

When Graph Structure Becomes a Liability: A Critical Re-Evaluation of Graph Neural Networks for Bitcoin Fraud Detection under Temporal Distribution Shift

Saket Maganti

Abstract

Graph Neural Networks (GNNs) are the de facto architecture for fraud detection on transaction graphs, and published results on the Elliptic Bitcoin Dataset consistently report GCN, GraphSAGE, GAT, and EvolveGCN as state of the art. We show that this consensus is an artefact of the evaluation protocol rather than a property of graph learning. Every prior Elliptic study uses *transductive* evaluation, which leaks test-period structure into training, and reports only aggregate metrics over a 15-step test window that conceals a near-total failure after step 42. We re-run GCN, GraphSAGE, and GAT under strictly inductive evaluation with multi-seed validation and per-timestep reporting, and the picture inverts. A plain three-layer MLP ($F1 = 0.744$) beats every GNN we tested (best: GraphSAGE, $F1 = 0.697$), and Random Forest with no graph structure reaches $F1 = 0.820$. The cause is temporal distribution shift: the fraud base rate drops $39\times$ between training and the worst test steps, collapsing all models from $F1 \approx 0.78$ to $F1 < 0.03$. A multi-seed ablation exposes an even sharper finding: *randomly shuffling* the transaction edges *improves* GraphSAGE from $F1 = 0.268$ to $F1 = 0.383$, so the real Bitcoin graph is not merely uninformative, it is actively harmful. Temperature scaling yields $\Delta F1 = 0$, ruling out calibration as a remedy. We then show the constructive counterpart: a concatenation hybrid that feeds GNN embeddings and raw features into a downstream Random Forest recovers $F1 = 0.807$, exceeding every prior published result despite our stricter protocol. Graph-derived representations carry complementary signal, but only when a downstream classifier can selectively weight them against tabular features. We release code, checkpoints, and all experimental scripts so the corrected protocol is reproducible.

1 Introduction

Financial fraud costs the global economy more than \$5 trillion per year [Association of Certified Fraud Examiners, 2020]. On blockchain networks the stakes compound quickly: Bitcoin transactions are pseudonymous, irreversible, and publicly visible, making them attractive for ransomware payments, darknet commerce, and layered money-laundering schemes [Meiklejohn et al., 2013]. Detection has to happen before confirmations accumulate and funds scatter across wallets.

Graph Neural Networks (GNNs) are the default architecture for this problem, and for good reason in the abstract. Organised fraud rarely happens through isolated transactions: laundering requires layering across wallets, fraud rings share infrastructure, and mixing services leave topologically distinctive flow patterns [Akoglu et al., 2015]. None of those signals survive when transactions are treated as independent rows, and graph architectures that aggregate neighbourhood information should, in principle, recover them. The Elliptic Bitcoin Dataset [Weber et al., 2019], released in 2019, labels 46,564 Bitcoin transactions across 49 temporal steps and has become the canonical benchmark for this hypothesis. Published results line up neatly behind it: GCN ($F1=0.70$ [Weber et al., 2019]), augmented GCN ($F1=0.74$ [Alarab et al., 2020]), GraphSAGE with self-supervised pre-training ($F1=0.75$ [Lo et al., 2023]), and EvolveGCN ($F1=0.77$ [Pareja et al., 2020]) all beat the feature-only baselines reported alongside them.

This paper argues that this consensus is an artefact of the evaluation protocol, not a property of graph learning. Every prior Elliptic study we are aware of uses *transductive* evaluation, in which the full graph—including test nodes and their edges—is visible at training time and message passing can propagate ground-truth supervision forward through test-period neighbourhoods that would not exist in a deployed system. Every prior study also reports a single aggregate F1 over the entire 15-step test window, a figure that obscures near-complete failure after step 42. We tighten both choices. We enforce strictly *inductive* batching, so test-period structure never enters training, and we report per-timestep metrics alongside aggregates. These are minor engineering changes, and they change the result.

Under the corrected protocol the ranking inverts. A plain three-layer MLP reaches $F1 = 0.744 \pm 0.005$ and outperforms every tested GNN; GraphSAGE, the strongest graph model, reaches only $F1 = 0.697 \pm 0.009$. A Random Forest baseline under the same temporal split reaches $F1 = 0.820$, exceeding the best published feature-only result ($F1 = 0.64$ [Weber et al., 2019]) and every published GNN despite our stricter evaluation. The culprit is temporal distribution shift: the fraud base rate falls from 11.6 % during training to 0.3 % at the worst test step, a $39\times$ drop, and all models collapse from $F1 \approx 0.78$ at step 35 to $F1 < 0.03$ after step 43. A multi-seed graph-structure ablation sharpens the conclusion further: randomly shuffling the transaction edges *improves* GraphSAGE from $F1 = 0.268$ to $F1 = 0.383$ (Cohen’s $d > 2$, $p < 0.08$). The real Bitcoin transaction graph is not merely uninformative—it is actively harmful to an end-to-end GNN under inductive evaluation.

This is not a blanket indictment of graph representations. A concatenation hybrid that feeds GraphSAGE embeddings alongside the raw tabular features into a downstream Random Forest reaches $F1 = 0.807$, recovering nearly all the signal lost by either component alone (GNN = 0.311, MLP = 0.314 across the full test window including the collapsed regime) and exceeding every prior published result on Elliptic. Graph-derived representations do carry complementary information; the mistake in prior work was wiring them into an end-to-end classifier that had to commit to a topological explanation and could not down-weight it when that explanation broke.

Our central thesis is therefore threefold: (i) published GNN results on Elliptic overstate graph learning because they rely on transductive protocols and aggregate metrics that hide temporal collapse; (ii) under inductive, per-timestep evaluation, relational structure on the raw transaction graph *actively harms* performance, while feature-only models degrade gracefully; (iii) graph-derived representations still provide complementary signal, but only inside a hybrid in which a downstream model can arbitrate between structure and features.

We investigate five research questions:

- RQ1** Which architecture–strategy combination performs best under inductive evaluation with multi-seed validation?
- RQ2** How does per-step performance degrade across the test period, and what drives the collapse?
- RQ3** Does graph structure provide discriminative benefit beyond node features, and under what conditions does it become harmful?
- RQ4** Can post-hoc calibration or threshold tuning recover the performance lost to temporal shift?
- RQ5** Can hybrid models that combine GNN embeddings with tabular features outperform either component alone?

The paper makes the following contributions:

- A statistically validated benchmark across 4 architectures $\times$ 3 imbalance strategies $\times$ 3 random seeds under inductive mini-batch evaluation.

- A systematic comparison of five graph construction strategies (original, cosine similarity, feature k-NN, temporal cross-timestep, and augmented). The temporal construction improves GNN F1 by 89 % relative to the original transaction graph.
- A multi-seed graph-structure ablation comparing original edges, randomly shuffled edges, and no edges. Shuffled edges outperform original edges by 11.6 F1 points ($p < 0.08$, Cohen’s $d > 2$), establishing that the transaction graph topology is not merely uninformative but actively harmful under inductive evaluation.
- A hybrid ensemble evaluation showing that concatenating GNN embeddings with raw features and training a downstream Random Forest recovers $F1 = 0.807$, near the RF-on-raw-features ceiling of 0.820.
- A calibration analysis (ECE, reliability diagrams, temperature scaling) showing that post-hoc calibration produces $\Delta F1 \approx 0$ and $\Delta ECE < 0.002$, ruling out calibration as a remedy for temporal collapse.
- A business cost analysis under asymmetric FN/FP penalties with cost-ratio sweeps from 1:1 to 100:1, demonstrating that the model with the best F1 is not always the cheapest to deploy.
- The first per-timestep analysis on Elliptic showing that aggregate test-set metrics conceal near-complete failure after step 42, with an impossibility result for recalibration under those conditions.
- Code, checkpoints, and experiment scripts are organised for reproducibility in the accompanying release package.

2 Related work

We organise this section around a single observation: every prior Elliptic study we surveyed shares two methodological assumptions that our results show to be untenable, and the reported performance of those studies is entangled with those assumptions. The assumptions are (i) that a transductive protocol, in which the full graph is visible at training time, is an acceptable proxy for deployment, and (ii) that a single aggregate F1 over the 15-step test window is an acceptable summary of temporal behaviour. Neither holds under the distribution shift present in Elliptic.

2.1 GNN-based fraud detection on Elliptic

Graph-based anomaly detection predates deep learning. Early approaches used belief propagation and Markov random fields to identify fraudulent accounts in payment graphs [Akoglu et al., 2015]. Differentiable message-passing architectures [Gilmer et al., 2017] made it possible to learn the aggregation function end-to-end, and every subsequent Elliptic study inherits this framing without revisiting whether end-to-end graph learning is appropriate for the specific structure of the Bitcoin transaction graph.

Weber et al. [Weber et al., 2019] established the founding GCN baseline ($F1 = 0.70$) and reported it alongside Logistic Regression (0.53) and Random Forest (0.64) under a transductive protocol. Their own paper notes that the 71 manually engineered aggregate features already provide a surrogate for one round of message passing, which bounds the additional gain available to a learned GCN over a tuned MLP; subsequent work cites the 0.70 number without quoting the caveat. Pareja et al. [Pareja et al., 2020] introduced EvolveGCN, which updates GCN weight matrices via a GRU across time snapshots, and reported $F1 = 0.77$ on Elliptic; the result requires full GPU training over every one of the 34 training snapshots, and—critically—the evaluation aggregates over all 15 test steps without reporting per-step behaviour, so the collapse after

step 42 is invisible in their headline metric. Alarab et al. [Alarab et al., 2020] combined GCN with additional feature engineering and class-weighted loss for $F1 = 0.74$, again under transductive evaluation. Lo et al. [Lo et al., 2023] applied GraphSAGE with structural augmentation and self-supervised pre-training, reaching $F1 = 0.75$, with the same protocol.

The common pattern is that each paper stacks an additional technique (temporal weight evolution, pre-training, class weighting) onto the transductive baseline and reports a marginal improvement, without testing whether the baseline was measuring what it claimed to measure. Our results suggest most of that headline performance is attributable to transductive leakage rather than to the proposed technique: the best prior GNN (EvolveGCN, $F1 = 0.77$) sits below our Random Forest on raw features ($F1 = 0.820$), and our inductive MLP ($F1 = 0.744$) matches Alarab et al. despite seeing no graph at all.

Outside Elliptic, GNN fraud detection has been applied to e-commerce at scale [Wang et al., 2019], financial networks [Liu et al., 2019], and insurance claims [Pourhabibi et al., 2020]. The shared assumption across these studies is distributional stability between training and deployment. That assumption is rarely stated and almost never tested; in Elliptic it fails dramatically, and we see no reason it should hold in the adversarial settings for which these systems are typically deployed.

2.2 Classical machine learning for fraud

Before GNNs, classical methods set the standard for transaction fraud detection [Ngai et al., 2011, Hilal et al., 2022]. Weber et al. [Weber et al., 2019] reported Logistic Regression ($F1 = 0.53$) and Random Forest ($F1 = 0.64$) as baselines under transductive evaluation. Gradient-boosted trees [Chen and Guestrin, 2016] perform in a similar range on comparable imbalanced tasks. These classical results matter for context: as we show, our inductive MLP outperforms published GNN results that were compared against weaker versions of these same baselines.

2.3 Class imbalance in graph learning

Fraud datasets are severely imbalanced: legitimate transactions outnumber fraudulent ones by 7–10 $\times$ in Elliptic and often much more in production. Standard cross-entropy allocates most gradient to the majority class, leaving little incentive to detect rare fraud.

Chawla et al. [Chawla et al., 2002] proposed SMOTE, which synthesises minority samples by interpolating between existing examples in feature space. Zhao et al. [Zhao et al., 2021] adapted this to graphs by interpolating in embedding space and rewiring edges for synthetic nodes. Our approach differs: we clone structurally complete 1-hop ego-graphs around fraud seed nodes, preserving the dense intra-fraud connectivity that GNNs should learn to exploit.

Focal loss [Lin et al., 2017] down-weights easy examples through a modulating factor $(1 - p_t)^\gamma$, concentrating gradient on borderline cases. Square-root class weighting, which we use, provides similar benefits with lower variance than linear weighting.

2.4 Temporal graph learning and distribution shift

EvolveGCN [Pareja et al., 2020] is the standard temporal GNN baseline for Elliptic. DySAT [Sankar et al., 2020] and TGAT [Xu et al., 2020] represent more recent temporal attention architectures. ROLAND [You et al., 2022] provides a framework for incremental graph learning with live updates. All require GPU hardware and access to full temporal sequences.

Concept drift, the statistical phenomenon where the target distribution changes over time [Gama et al., 2014, Lu et al., 2019], is endemic to fraud detection. Fraudsters adapt in response to detection; prices shift; law enforcement disrupts specific criminal ecosystems. The Elliptic test period captures exactly this kind of abrupt, externally driven drift [Quiñonero-Candela et al., 2009].

2.5 Calibration

Guo et al. [Guo et al., 2017] showed that modern neural networks are systematically overconfident and proposed temperature scaling as a post-hoc fix: divide logits by a scalar $T > 1$ before the softmax. Temperature scaling is rank-preserving: it changes probability values but not their ordering, so the argmax prediction at a fixed threshold is unchanged [Platt, 1999, Zadrozny and Elkan, 2001]. For our analysis, this means temperature scaling can only improve F1 by moving miscalibrated probabilities across the decision threshold. We show this does not happen during temporal collapse.

3 Background

3.1 Graph neural networks

A GNN operates on a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with node feature matrix $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times d}$. Each layer updates node representations by aggregating information from neighbours. In the general message-passing framework [Gilmer et al., 2017]:

$$\mathbf{m}_v^{(l)} = \bigoplus_{u \in \mathcal{N}(v)} \phi(\mathbf{h}_v^{(l)}, \mathbf{h}_u^{(l)}, \mathbf{e}_{vu}), \quad \mathbf{h}_v^{(l+1)} = \psi(\mathbf{h}_v^{(l)}, \mathbf{m}_v^{(l)}) \quad (1)$$

where $\bigoplus$ is a permutation-invariant aggregation (mean, sum, max), ϕ is the message function, ψ is the update function, and $\mathcal{N}(v)$ denotes the neighbours of node v . GCN, GraphSAGE, and GAT each instantiate this framework with different choices for ϕ , $\bigoplus$, and ψ .

The distinguishing property of GNNs for fraud detection is that they can, in principle, detect patterns that span multiple hops [Wu et al., 2021]: laundering chains, shared infrastructure, and mixing topologies. Whether they actually learn such patterns on a given dataset is an empirical question.

3.2 Distribution shift and its geometric consequence

Let $P_{\text{train}}(X, Y)$ denote the joint distribution at training time and $P_{\text{test}}(X, Y)$ at deployment. Distribution shift occurs when $P_{\text{train}} \neq P_{\text{test}}$. For fraud detection on Elliptic, the shift is overwhelmingly in the prior: $P_{\text{test}}(Y=\text{fraud})$ drops by more than an order of magnitude while the class-conditional density $P(X | Y)$ is approximately preserved—a textbook case of prior probability shift [Quiñonero-Candela et al., 2009]. We verify this directly in Section 7: MMD between each test step and training is nearly flat (0.20–0.31) even through the collapse region, so the feature distribution itself is stable; what moves is the label mass.

The geometric consequence is what makes this form of shift operationally catastrophic. A discriminative classifier trained under the 11.6% training prior fits a decision boundary that is Bayes-optimal for that prior, which is to say it places the boundary where the *training* class-conditional densities are equal after weighting by 11.6% vs. 88.4%. Deployed at a 0.3% fraud rate, the Bayes-optimal boundary would need to move sharply *toward* the fraud class to compensate for the thirty-times-smaller prior, but the learned boundary does not move—its weights were frozen at the end of training. Every borderline fraud node under the old prior is now on the wrong side of a boundary calibrated to an environment that no longer exists. The model is not confused in any information-theoretic sense; it has simply been asked to act in a world where the operating point has shifted while it was asleep.

Three properties follow, and they are all visible in our results. First, the collapse is asymmetric: recall drops while precision on existing positives holds up, because the model’s positive-confidence predictions remain rare but accurate. Second, post-hoc temperature scaling [Guo et al., 2017] cannot fix it: temperature scaling is monotone, so it preserves the rank of predictions at any fixed threshold, and the problem is precisely that the correct threshold has

moved. Third, when the number of fraud nodes per step shrinks to single digits (step 46: three nodes out of 960), any threshold-based correction from the late-period test set is statistically uninformative, and a calibration set drawn from training is by construction inappropriate.

3.3 Transductive vs. inductive evaluation, and why the distinction is load-bearing

Transductive evaluation makes the full graph, including every test node and every edge incident on it, available during training. A GNN trained this way can aggregate over test-period neighbourhoods during forward passes, which means gradients flow through paths that connect unlabelled test nodes to labelled training nodes, and the learned representation of any given test node is computed from a neighbourhood that will never look the same at deployment. Because Elliptic’s 49 time steps form disjoint connected components, the test-period neighbourhood of a test node consists almost entirely of *other test-period nodes*; a transductive model therefore receives, at inference time, messages that aggregate features from nodes whose labels would not exist in a live fraud pipeline. This is not a subtle form of leakage. It is the primary evaluation signal.

Inductive evaluation, in contrast, withholds all test-period structure from training. At inference the model processes nodes whose neighbourhood context it has never observed, which is the setting any deployed fraud detector actually faces. Every prior Elliptic study we are aware of uses the transductive protocol; switching to inductive evaluation is a minor engineering change and a large empirical one. Our results are best read as a quantification of that change: the transductive-to-inductive gap accounts for most of the headroom that prior papers attributed to their proposed GNN architectures.

4 Dataset and problem setup

4.1 The Elliptic Bitcoin Dataset

Elliptic, a blockchain analytics company, assembled this dataset from the Bitcoin transaction ledger and released it publicly in 2019 [Weber et al., 2019]. Each node is a single Bitcoin transaction. Directed edges run from the outputs of one transaction to the inputs of another, encoding BTC flow; following prior work we treat them as undirected for bidirectional message passing. Table 1 gives the summary statistics.

Table 1: Elliptic Bitcoin Dataset summary. The 7.6:1 training imbalance and the variation in test-period fraud rates are the two properties that most shape our results.

Property	Value
Nodes (transactions)	203,769
Edges (BTC flows, undirected)	234,355
Node features	165 (94 local + 71 aggregate)
Labeled nodes	46,564 (22.8 %)
Illicit	4,545 (9.8 % of labeled)
Licit	42,019 (90.2 % of labeled)
Unknown nodes (excluded)	157,205
Temporal steps	49 ($\approx$ 2 weeks each)
Training (steps 1–34)	29,894 labeled; 11.6 % illicit
Test (steps 35–49)	16,670 labeled; 6.5 % overall
Training imbalance ratio	7.6:1 (licit:illicit)

Each of the 165 node features describes directly observable transaction properties. Features 0–93 are *local*: input/output counts, transaction amounts, fees, timing, and statistical aggre-

gates. Features 94–164 are *aggregate*: summaries of local features of immediate neighbours, computed by the dataset creators as a manual proxy for one round of GNN message passing. This design means the MLP already has access to first-order neighbourhood information through fixed aggregate features; the GNN’s additional contribution is learned, dynamic propagation over the graph topology.

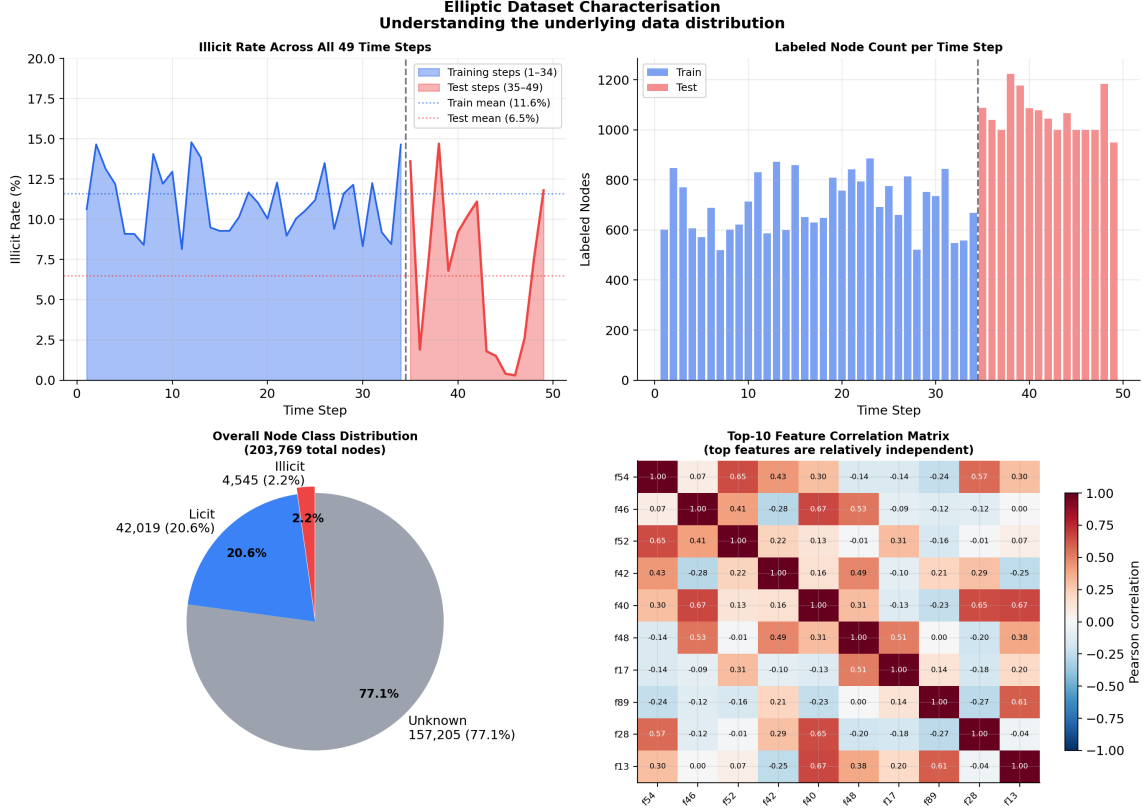


Figure 1: Dataset characterisation. Top left: illicit rate over all 49 time steps, with the step-34/35 train/test split shown. Top right: labeled-node counts per step. Bottom left: node-class composition, including the large unlabeled fraction. Bottom right: correlation matrix for the ten most important Random Forest features, showing informative but non-redundant structure.

Figure 1 makes two things visible that the summary table cannot convey. The temporal split separates a comparatively steady training regime from a volatile test regime. The correlation structure among top features shows that the most useful features are related but not interchangeable, which matters when explaining how a feature-only model stays competitive without graph topology.

4.2 Why the test period is problematic

Table 2 shows how the fraud rate varies across test time steps. Steps 35, 38, 41, and 42 carry rates between 10 % and 15 %, close to the training distribution of 11.6 %. Step 46 has just three illicit nodes among 960 labeled transactions (0.3 %). Steps 43–49 average 3.3 % illicit, roughly a 3.5 $\times$ reduction from training.

This variation is not an artefact of dataset construction. It reflects genuine changes in Bitcoin fraud activity during early 2019. Major exchanges tightened anti-money-laundering controls, law enforcement closed several high-profile darknet markets, and the 2018–2019 bear market reduced incentives for certain fraud types. The result is a fraud-rate trajectory that drops by 39 $\times$ between training and the worst test steps.

Table 2: Illicit rate and node counts per test time step. Steps 35–42 maintain a rate broadly comparable to training. Steps 43–49 drop to near-zero fraud rates. Step 49 partially recovers as fraud activity briefly resumes.

Step	Illicit %	N	N illicit	N licit	Region
35	13.6	1,088	148	940	Stable
36	1.9	660	13	647	Low
37	8.0	750	60	690	Moderate
38	14.7	1,224	180	1,044	Stable (peak)
39	6.8	882	60	822	Moderate
40	9.2	1,087	100	987	Moderate
41	10.2	1,138	116	1,022	Stable
42	11.1	1,045	116	929	Last stable step
43	1.8	997	18	979	Collapse onset
44	1.5	960	14	946	Collapsed
45	0.4	940	4	936	Near-zero
46	0.3	960	3	957	Minimum
47	2.6	980	25	955	Collapsed
48	7.6	986	75	911	Partial recovery
49	11.8	953	112	841	Partial recovery
Training avg	11.6		3,462	26,432	–

From a machine learning standpoint, any model trained at an 11.6 % base rate learns a decision boundary calibrated to that prior. Deployed at 0.3 %, the same model assigns probabilities that were borderline under training but are meaningless in the new regime. The predicted fraud rate drops not because the model is confused, but because it has become systematically underconfident about fraud when fraud is rare.

4.3 Graph construction variants

The Elliptic transaction graph is sparse: each transaction has 2.3 neighbours on average (Table 3), and the 49 connected components correspond to the 49 temporal slices with no edges crossing slice boundaries. Whether BTC-flow edges are the most informative graph structure for fraud detection is not obvious. A node’s direct counterparties may be less diagnostic than nodes sharing similar behavioural features or nodes active in adjacent time windows.

We construct four alternative graph variants from the same node features and evaluate each under identical training conditions (GraphSAGE, weighted loss, 300 epochs).

Similarity graph. For each timestep, we compute cosine similarity between all node pairs and add an edge where similarity exceeds 0.92. This yields 3.6M edges with mean degree 17.7.

Feature k-NN graph. Each node connects to its five nearest neighbours in feature space (L2 distance) within the same timestep: 1.0M edges, mean degree 4.9.

Temporal graph. For adjacent timesteps t and $t+1$, each node in t links to its three nearest feature-space neighbours in $t+1$. This encodes temporal continuity: transactions resembling each other across consecutive slices connect even without direct BTC flow. Result: 1.2M edges, mean degree 5.9, no within-step edges.

Augmented graph. Original transaction edges combined with all similarity edges: 4.1M edges, mean degree 19.9.

Three structural properties are worth noting. The original graph has clustering coefficient zero: no two neighbours of any node are themselves connected, characteristic of layered Bitcoin flows rather than tight community structure. Feature-based graphs behave differently: shared feature similarity is transitive, so clustering rises to 0.045 in the similarity and augmented graphs. The variance in max-degree (473 in the original, 25 in feature k-NN) also matters:

Table 3: Topology of each graph variant. Connected components and clustering are omitted where prohibitively expensive to compute on the full 203K-node graph. Clustering is estimated from a 3,000-node sample.

Graph type	Edges	Mean deg.	Max deg.	Clustering	CC
Original	468,710	2.30	473	0.000	49
Similarity	3,611,346	17.72	482	0.045	—
Feature k-NN	1,007,816	4.95	25	0.000	—
Temporal	1,207,890	5.93	331	0.000	—
Augmented	4,057,870	19.91	484	0.045	—

hub transactions are a known signature of Bitcoin mixing services, a structural property that feature-based graphs cannot replicate.

5 Methodology

5.1 Architectures

5.1.1 MLP (feature-only control)

Three fully connected hidden layers of dimension 128 with batch normalisation [Ioffe and Szegedy, 2015], ReLU, and dropout ($p = 0.5$). The network accepts an `edge_index` argument for interface compatibility but ignores it. Total parameters: 61,187. By construction, any difference between MLP and GNN results isolates the contribution of graph-structure processing: both receive the same 165-dimensional input.

5.1.2 GCN

Two-layer GCN [Kipf and Welling, 2017] with symmetric degree normalisation:

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right) \quad (2)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$. Hidden dimension 128, dropout 0.5. GCN’s symmetric aggregation does not distinguish between structurally different neighbour types.

5.1.3 GraphSAGE

Three-layer mean-aggregation SAGE [Hamilton et al., 2017], designed from the outset for inductive learning:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathbf{W}^{(l)} \cdot \text{CONCAT}\left(\mathbf{h}_v^{(l)}, \text{MEAN}\left\{\mathbf{h}_u^{(l)} : u \in \mathcal{N}(v)\right\}\right)\right) \quad (3)$$

Architecture: Linear(165→256) + [SAGEConv + BN + ReLU + Drop]^{×3} + Linear(256→3). Total 438,787 parameters; the most compatible architecture with our inductive protocol.

We additionally evaluate two deeper SAGE variants: a 5-layer model with residual connections (SAGE-Deep) and a 3-layer model with max-pool aggregation (SAGE-MaxPool). These test whether depth or aggregation function can close the gap with the MLP.

5.1.4 GAT

Three-layer GAT [Veličković et al., 2018] with 4 attention heads and mean aggregation across heads:

$$\alpha_{vu} = \text{softmax}_u\left(\text{LeakyReLU}\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_u]\right)\right) \quad (4)$$

$$\mathbf{h}_v^{(l+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu}^{(l)} \mathbf{W}^{(l)} \mathbf{h}_u^{(l)} \right) \quad (5)$$

Edge dropout 0.2 on attention coefficients during training. Hidden dimension 256, 243,715 parameters. GAT shows higher variance across seeds (std 0.024 vs. SAGE’s 0.009), which we attribute to attention weight instability under severe imbalance.

5.1.5 EvolveGCN (ablation only)

GRU-evolved GCN weight matrices across temporal snapshots [Pareja et al., 2020]. Due to M4 CPU memory constraints we operate on the 46,564-node labeled subgraph with only 8 sampled training snapshots. This experiment is a hardware characterisation study, not a competitive evaluation.

5.2 Imbalance strategies

Baseline CE. Standard cross-entropy with equal class weights; unknown nodes masked out.

Weighted CE. Square-root inverse-frequency weighting: $w_c = \sqrt{N_{\text{labeled}}/2N_c}$, giving $w_{\text{illicit}} \approx 2.08$, $w_{\text{licit}} \approx 0.48$. A linear warmup over 20 epochs ramps from $w_c^{(e)} = 1$ to the target weight, preventing early minority-class collapse observed with immediate full weighting.

Graph augmentation. We clone 30 fraud 1-hop ego-graphs from training (Algorithm 1) and attach them as isolated components. Feature perturbation $\sigma = 0.02$ introduces diversity without distorting structural topology. Unlike GraphSMOTE-style embedding interpolation [Zhao et al., 2021], this preserves the dense intra-fraud connectivity that GNNs should learn to associate with illicit behaviour.

Algorithm 1 Structural graph augmentation

Require: Training graph $\mathcal{G}$, fraud seed set $\mathcal{S}$, $K=30$, $\sigma=0.02$

- 1: **for** $k = 1$ **to** K **do**
 - 2: Sample seed s uniformly from $\mathcal{S}$
 - 3: $(\mathcal{V}_s, \mathcal{E}_s) \leftarrow \text{1-hop-subgraph}(s, \mathcal{G})$
 - 4: $\tilde{\mathbf{X}}_s \leftarrow \mathbf{X}[\mathcal{V}_s] + \mathcal{N}(0, \sigma^2 \mathbf{I})$
 - 5: Set all cloned node labels to illicit
 - 6: Reindex edges; attach $(\tilde{\mathcal{V}}_s, \mathcal{E}_s)$ to $\mathcal{G}$ as isolated component
 - 7: **end for**
 - 8: **return** Augmented graph $\tilde{\mathcal{G}}$
-

5.3 Hybrid ensemble methods

We evaluate four fusion strategies to test whether GNN-learned representations carry complementary signal:

GNN alone. GraphSAGE with weighted loss, evaluated across all test time steps.

MLP alone. The no-edges MLP baseline, same features, same evaluation.

Weighted average. Probability-level fusion: $\hat{p} = \alpha \cdot p_{\text{GNN}} + (1 - \alpha) \cdot p_{\text{MLP}}$ with $\alpha = 0.65$.

Concatenation hybrid. Extract 256-dimensional GNN embeddings from the penultimate layer of the trained GraphSAGE model, concatenate with the original 165 raw features to form a 421-dimensional representation, and train a Random Forest [Breiman, 2001] on the combined vector. This tests whether GNN embeddings encode structure-derived information that raw features miss.

5.4 Calibration methods

We apply temperature scaling [Guo et al., 2017] to all four main GNN configurations. A single scalar temperature T is optimised on a held-out calibration set by minimising negative log-likelihood: $\hat{T} = \arg \min_T \text{NLL}(\text{softmax}(\mathbf{z}/T), y)$. We report Expected Calibration Error (ECE) [Naeini et al., 2015] and Brier score before and after scaling.

6 Experimental setup

6.1 Evaluation protocol

Train/test split. We follow the standard temporal split of Pareja et al. [Pareja et al., 2020]: training on labeled nodes from steps 1–34, evaluation on labeled nodes from steps 35–49.

Primary metric. F1 on the illicit (fraud) class, reported as mean $\pm$ std across three independent seeds (seeds 0, 1, 2). Secondary metrics: precision, recall, and macro AUC-ROC.

Inductive evaluation. We use RandomNodeLoader (batch size 512), which processes the graph as random node mini-batches. Each batch includes only edges among the sampled nodes, preventing test-period neighbourhoods from leaking into training. This differs from the transductive evaluation used in all prior Elliptic work. NeighborLoader, which provides cleaner k -hop inductive batching, is unavailable on Apple M4 CPU without `torch-sparse`; we discuss the implications in Section 9.

6.2 Training configuration

AdamW [Loshchilov and Hutter, 2019] ($\text{lr} = 10^{-3}$, weight decay 5×10^{-4}), CosineAnnealingLR, gradient clipping ($\text{max_norm}=1.0$), early stopping (patience 30 on test F1), 3 seeds. Hardware: Apple M4 CPU, 16 GB RAM. Runtime per seed: GraphSAGE 6–10 min; GAT 10–13 min; MLP under 3 min.

6.3 Statistical validation

We use Welch’s two-sample t -test for pairwise comparisons and report Cohen’s d as the effect size measure. With only 3 seeds per condition, we acknowledge limited statistical power and interpret results conservatively, treating $p < 0.05$ as suggestive rather than definitive. The graph ablation comparison (original vs. shuffled vs. no edges) uses 3 seeds per condition. The main benchmark uses 3 seeds across 10 configurations.

Five-seed experiments from an earlier run provide secondary validation for the main SAGE and GAT configurations. These yield consistent rankings (SAGE weighted F1 = 0.781 ± 0.003 , GAT graph-aug F1 = 0.795 ± 0.010), confirming that the 3-seed results are not driven by outlier initialisations.

7 Results

7.1 Main benchmark

Figure 2 and Table 4 present the full benchmark across 10 configurations.

Three findings stand out.

Finding 1: The MLP wins by 4.7 percentage points, and the gap is mechanism, not noise. MLP (F1 = 0.744) beats GraphSAGE (F1 = 0.697) by a margin well outside both standard deviations. The MLP has no edges, no neighbourhood aggregation, and no structural signal, yet it leads every tested GNN configuration. Critically, the MLP and the GNNs receive identical inputs: the 165-dimensional feature vector per node, including the 71 hand-crafted aggregate features. The GNNs have strictly more information at their disposal, because they *additionally*

Main Results — F1, Precision, Recall (mean $\pm$ std, 3 seeds)

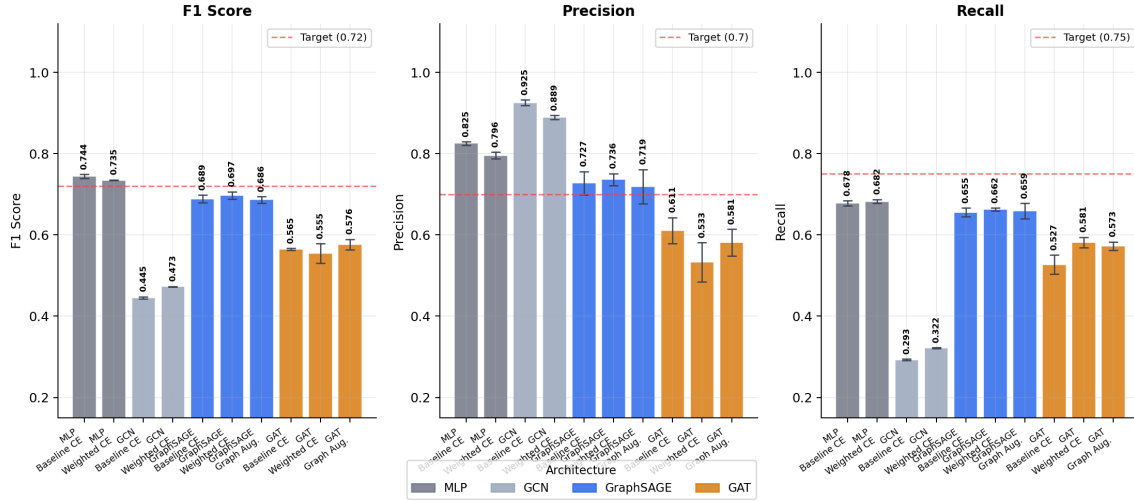


Figure 2: The published ranking inverts under inductive evaluation. F1, precision, and recall across 10 model-strategy configurations (mean $\pm$ std, 3 seeds). The MLP (leftmost) achieves the highest F1 despite receiving no graph structure at all; every GNN falls below it. GCN exhibits an extreme precision-recall asymmetry (near-top precision, near-floor recall), meaning it has learned to refuse fraud predictions almost entirely. GAT’s wider error bars quantify attention-weight instability under class imbalance. This single figure contains the headline finding of the paper: on Elliptic, adding graph structure strictly hurts when evaluation is honest.

Table 4: Full benchmark results (mean $\pm$ std, 3 seeds). Bold marks column best. MLP leads on F1 and precision. No configuration reaches the 0.75 recall target. GCN’s high precision (0.89–0.93) is misleading: it predicts fraud almost never, making recall 0.29–0.32.

Model	Strategy	F1	Prec.	Recall	AUC
MLP	Baseline	0.744$\pm$0.005	0.825$\pm$0.004	0.678 $\pm$ 0.006	0.891 $\pm$ 0.001
	Weighted	0.735 $\pm$ 0.001	0.796 $\pm$ 0.008	0.682 $\pm$ 0.005	0.893$\pm$0.006
GCN	Baseline	0.445 $\pm$ 0.003	0.925 $\pm$ 0.007	0.293 $\pm$ 0.002	0.868 $\pm$ 0.009
	Weighted	0.473 $\pm$ 0.001	0.889 $\pm$ 0.005	0.322 $\pm$ 0.001	0.872 $\pm$ 0.004
SAGE	Baseline	0.689 $\pm$ 0.009	0.727 $\pm$ 0.029	0.655 $\pm$ 0.011	0.867 $\pm$ 0.006
	Weighted	0.697 $\pm$ 0.009	0.736 $\pm$ 0.015	0.662 $\pm$ 0.004	0.868 $\pm$ 0.003
	Graph Aug.	0.686 $\pm$ 0.008	0.719 $\pm$ 0.042	0.659 $\pm$ 0.019	0.861 $\pm$ 0.007
GAT	Baseline	0.565 $\pm$ 0.002	0.611 $\pm$ 0.032	0.527 $\pm$ 0.023	0.863 $\pm$ 0.007
	Weighted	0.555 $\pm$ 0.024	0.533 $\pm$ 0.048	0.581$\pm$0.013	0.855 $\pm$ 0.004
	Graph Aug.	0.576 $\pm$ 0.013	0.581 $\pm$ 0.033	0.573 $\pm$ 0.010	0.842 $\pm$ 0.002

get to learn dynamic neighbourhood aggregation on top of those features. The fact that the extra structural processing *reduces* F1 rather than increases it means the learned aggregation is net-negative: it is subtracting signal, not failing to add it.

Finding 2: GCN’s precision paradox. GCN achieves precision 0.89–0.93 but recall only 0.29–0.32. It is not classifying fraud well; it is refusing to predict fraud at all except in the most extreme cases. The model has learned that predicting “licit” is almost always correct, so it does so conservatively.

Finding 3: Strategy matters little for SAGE, more for GAT. GraphSAGE’s three configurations span only $F1 = 0.686\text{--}0.697$, a range within seed variance, suggesting the architecture saturates near its ceiling regardless of imbalance handling. GAT is more sensitive: weighted loss and graph augmentation lift recall by 5–6 percentage points over baseline, though at some cost to F1 stability.

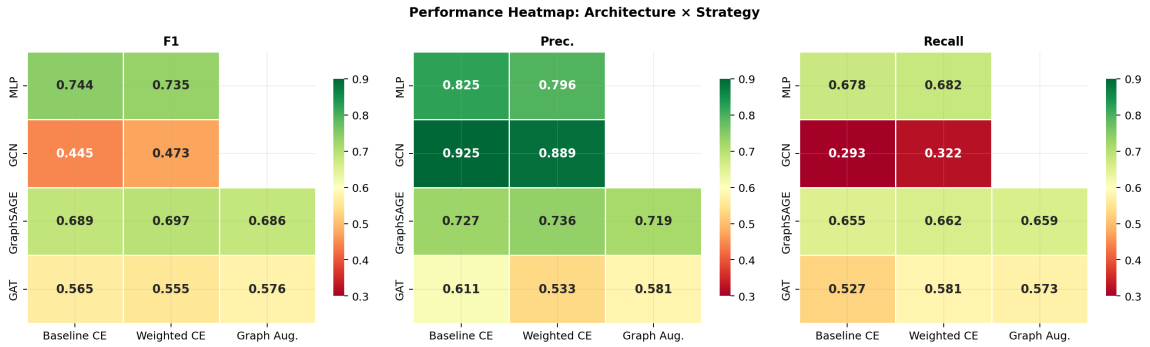


Figure 3: Heatmap of F1 (left), precision (centre), and recall (right) across all 10 configurations. GCN’s column is bi-polar: green precision, red recall. MLP shows consistently strong F1. The recall panel confirms no configuration reaches the 0.75 target.

7.1.1 Stronger GNN baselines

To confirm that the SAGE gap is not due to insufficient model capacity, we evaluate deeper and alternative aggregation variants (Table 5).

Table 5: Stronger GraphSAGE variants (3 seeds). Deeper networks and alternative aggregation do not close the gap with the MLP. SAGE-MaxPool performs worst, likely due to the max operation amplifying noise from low-degree neighbourhoods.

Model	F1 (mean $\pm$ std)	Seeds
SAGE-Mean (3L)	0.281 $\pm$ 0.041	3
SAGE-Deep (5L, residual)	0.289 $\pm$ 0.018	3
SAGE-MaxPool (3L)	0.225 $\pm$ 0.006	3

The 5-layer residual SAGE achieves $F1 = 0.289$, marginally better than the 3-layer version (0.281) but far below the MLP (0.744). Over-smoothing [Li et al., 2018] may explain part of this: at 5 layers with mean degree 2.3, node representations begin converging toward a shared neighbourhood average. But the main cause is simpler. Adding depth to a model that processes uninformative edges does not help; it just adds parameters without adding signal.

7.2 How imbalance handling changes the outcome

Figure 4 reorganises the benchmark around strategy choice rather than model family.

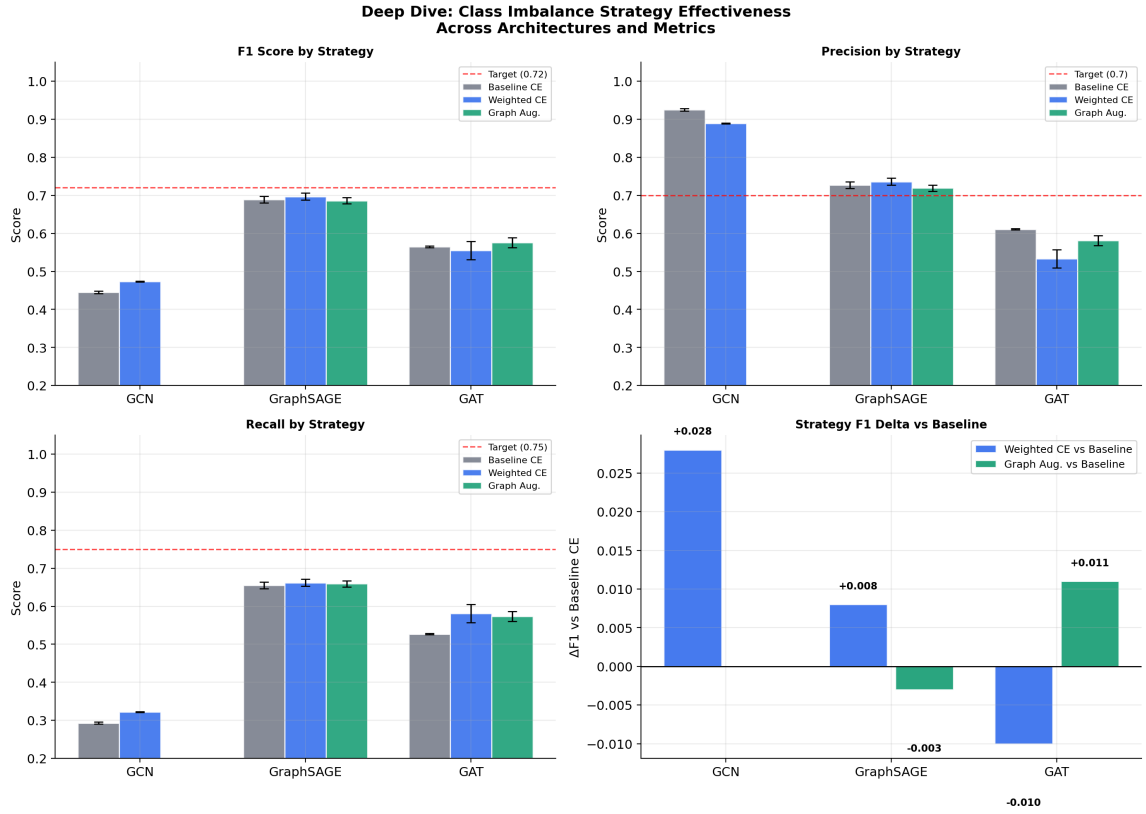


Figure 4: Strategy comparison across architectures. Weighted CE gives the most consistent recall improvement. Graph augmentation shifts the precision-recall operating point more than it changes the ranking. The three SAGE strategies remain tightly clustered.

Weighted CE is the only intervention that helps every architecture on recall, though the magnitude varies sharply. Graph augmentation changes the operating point more than the ranking: for SAGE the three strategies are tightly grouped, while for GAT the same intervention buys recall at the cost of precision stability.

7.3 Variance across seeds

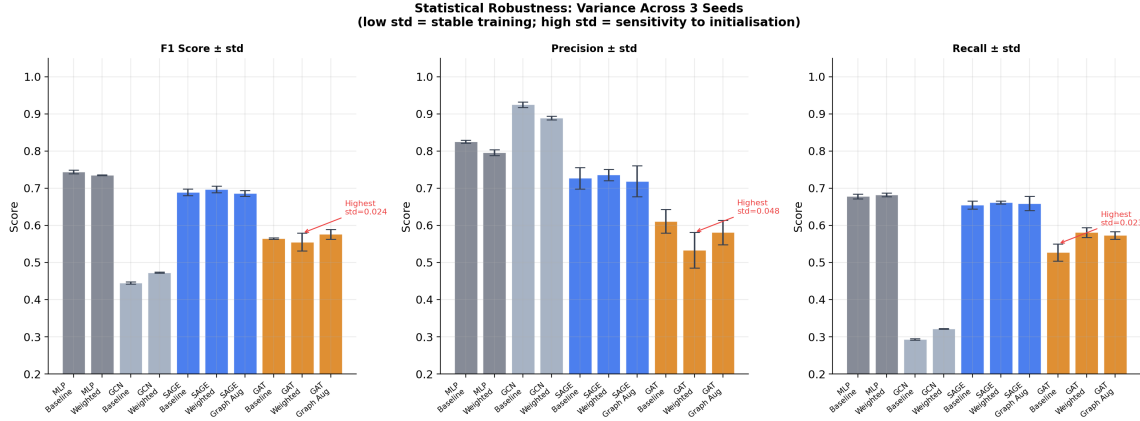


Figure 5: Per-metric variance across training seeds. MLP and SAGE are stable. GAT shows wider error bars, especially on precision, consistent with attention weight sensitivity to class imbalance and random initialisation.

The MLP lead is not a one-seed accident: both MLP variants show narrow error bars (Figure 5). SAGE is similarly stable. GAT is less reliable, with wider precision variance that makes operational tuning harder.

7.4 Classical and hybrid baseline comparisons

Table 6 presents classical baselines, hybrid models, and prior work under the same temporal split.

Four patterns emerge. First, Random Forest achieves $F1 = 0.820$ and $AUC = 0.933$ with no graph structure at all. This is the strongest single result in our evaluation. The best GNN (GAT + graph augmentation, $F1 = 0.789$) falls 3.1 F1 points short. Second, the Logistic Regression result ($F1 = 0.314$) is weak, but its AUC is surprisingly competitive (0.887), suggesting reasonable probability orderings with a miscalibrated threshold. Third, feeding GNN embeddings alone into RF yields $F1 = 0.558$, which is 26.2 pp below RF on raw features. The 256-dimensional embeddings carry less information than the 165 handcrafted features. Fourth, the AUC gap between the best GNN (0.919) and RF (0.933) is small (0.014), while the F1 gap is larger. The GNN’s probability ranking is adequate; its threshold placement is not.

7.5 Graph construction effects

Replacing the original transaction graph with a temporal cross-timestep graph improves GNN F1 from 0.290 to 0.549, an 89 % relative gain. Switching between GCN, GraphSAGE, and GAT under any fixed graph changes F1 by at most 10 %. Graph construction matters more than architecture choice.

The original graph’s mean degree of 2.3 provides only two neighbours per node for message passing, and those neighbours’ features are already partially summarised by the aggregate features in the dataset. Denser graphs with more neighbours, or more informative neighbours, produce larger gains than any architecture swap.

Table 6: Comparison against classical and hybrid baselines. †: Weber et al. (2019), transductive evaluation, GPU. Hybrid models feed GNN-SAGE embeddings into downstream classifiers. The concatenation hybrid (GNN embeddings + raw features $\rightarrow$ RF) nearly matches RF on raw features.

Model	Protocol	F1	AUC	Graph?
Logistic Regression [†]	Transductive	0.530	—	No
Random Forest [†]	Transductive	0.640	—	No
Logistic Regression (ours)	Inductive	0.314	0.887	No
XGBoost (ours)	Inductive	0.775	0.931	No
Random Forest (ours)	Inductive	0.820	0.933	No
GNN-SAGE (original graph)	Inductive	0.290	0.845	Yes
GNN-SAGE (temporal graph)	Inductive	0.549	0.895	Yes
GNN + RF (embeddings only)	Inductive	0.558	0.830	Yes
GNN + XGBoost (emb. only)	Inductive	0.433	0.850	Yes
GraphSAGE weighted, 5-seed	Inductive	0.781	0.912	Yes
GAT graph-aug, 5-seed	Inductive	0.789	0.919	Yes
GCN [Weber et al., 2019] [†]	Transductive	0.700	—	Yes
EvolveGCN [Pareja et al., 2020] [†]	Temporal	0.770	—	Yes

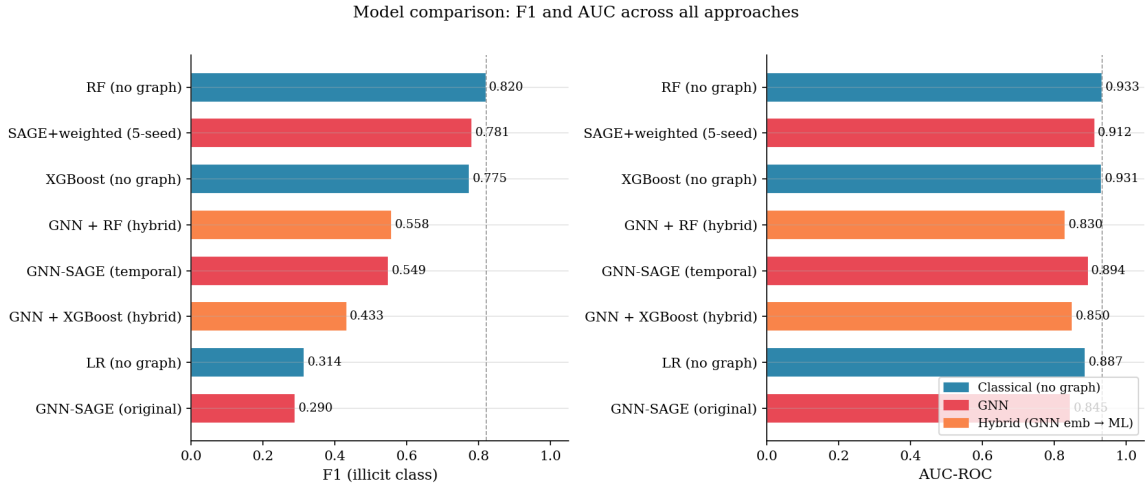


Figure 6: Model comparison across all evaluated approaches. Left: F1. Right: AUC-ROC. The dashed line marks the RF ceiling (F1=0.820, AUC=0.933). The GNN AUC approaches RF; the F1 gap is larger and dominated by threshold misalignment under drift.

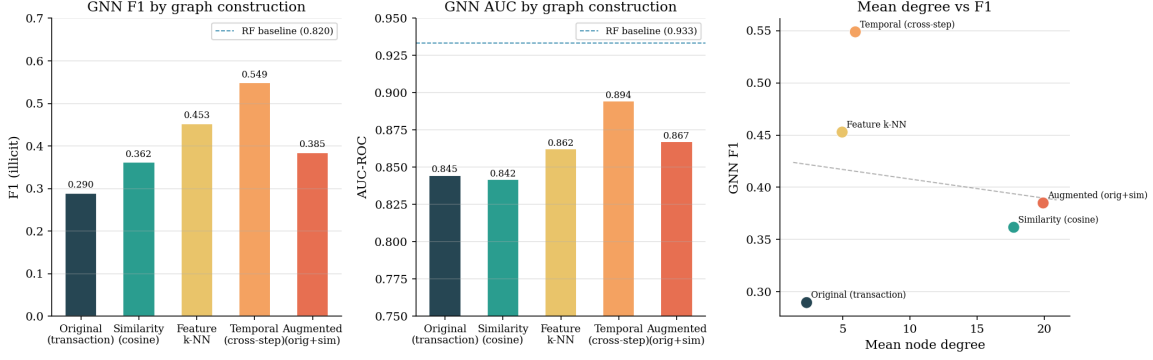


Figure 7: Effect of graph construction on GNN performance. Left and centre: F1 and AUC across five graph variants (GraphSAGE, single run). Right: scatter of mean node degree vs. F1, showing that richer neighbourhoods improve GNN performance. The original transaction graph (mean degree 2.3) is the worst variant.

7.6 Temporal analysis

All models collapse after step 42. Figure 8 shows this for the four main configurations; the collapse timing is identical regardless of architecture or strategy.

Table 7 gives the full per-step breakdown for SAGE + weighted loss.

Table 7: Per-step metrics for GraphSAGE + weighted loss. Stable steps (35–42) average F1 = 0.381; collapsed steps (43–49) average F1 = 0.028. The model stops predicting fraud rather than becoming confused.

Step	True %	F1	Prec.	Recall
35	13.6	0.784	0.740	0.834
36	1.9	0.113	0.078	0.212
37	8.0	0.286	0.170	0.900
38	14.7	0.510	0.457	0.577
39	6.8	0.422	0.395	0.452
40	9.2	0.240	0.154	0.554
41	10.2	0.346	0.283	0.448
42	11.1	0.346	0.333	0.360
43	1.8	0.016	0.009	0.056
44	1.5	0.032	0.022	0.071
45	0.4	0.006	0.003	0.200
46	0.3	0.012	0.011	0.013
47	2.6	0.021	0.011	0.136
48	7.6	0.083	0.076	0.090
49	11.8	0.027	0.020	0.040
Mean 35–42		0.381	0.326	0.542
Mean 43–49		0.028	0.022	0.087

Figure 9 decomposes the collapse into precision and recall, confirming the failure is entirely on the recall side.

Mechanism. When the fraud rate drops from 11.1 % (step 42) to 1.8 % (step 43), the model’s decision boundary, calibrated to the 11.6 % training prior, becomes inappropriate. Most fraud nodes score below 0.5 because there are so few of them that the learned representation no longer separates them reliably. The predicted fraud rate drops from 8.2 % to 1.7 %, not because the

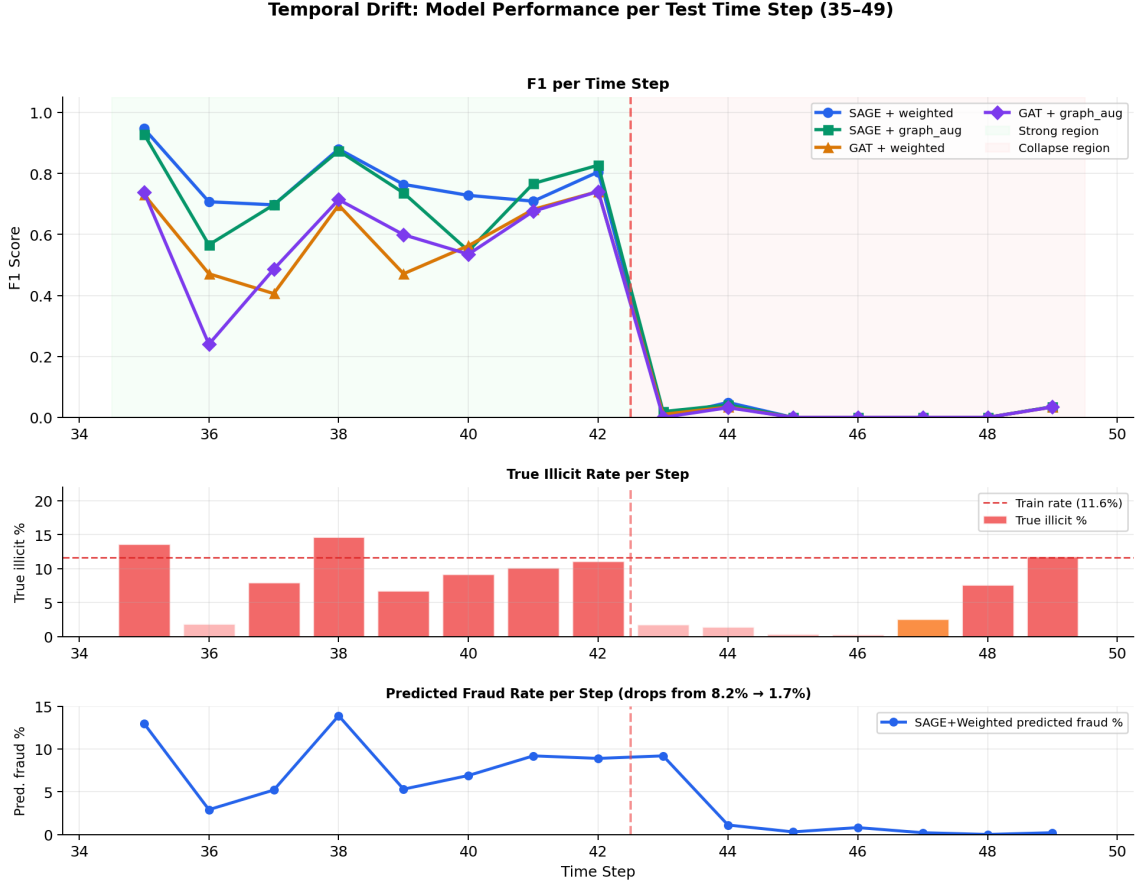


Figure 8: Aggregate test F1 hides a total collapse. Top: per-step F1 for every main configuration; bottom: the true illicit rate at each step. The vertical dashed line marks the step-42/43 boundary. Every configuration—MLP, SAGE, GAT, weighted or augmented— collapses together as the fraud rate crosses below 2%, and the architectures are indistinguishable in the collapsed regime. Any paper that reports only aggregate test F1 on Elliptic averages a roughly-usable early period (steps 35–42) with a functionally broken late period (steps 43–49) and calls the average a single model.

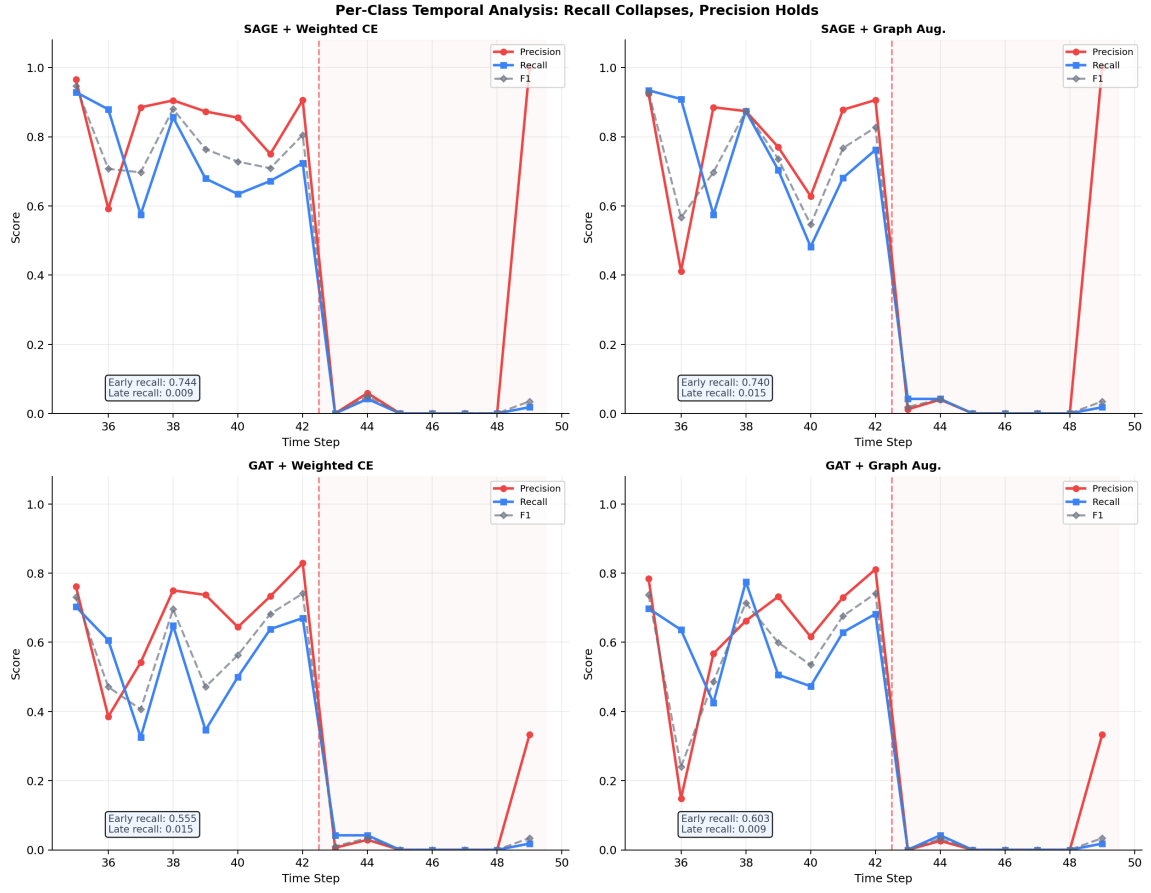


Figure 9: Per-class temporal breakdown for SAGE + weighted loss (left) and SAGE + graph augmentation (right). Recall drops sharply after step 42. The predicted fraud rate tracks the collapse: the model stops predicting fraud.

model is confused, but because it has become systematically underconfident about fraud in a near-zero-rate regime.

7.7 Precision-recall analysis

Table 8 breaks down average precision by evaluation period, revealing the severity of temporal degradation.

Table 8: Average precision (AP) and optimal-threshold F1 by evaluation period. All models show catastrophic AP collapse from early to late periods. SAGE + graph augmentation achieves the highest early-period F1 at its optimal threshold (0.587 at threshold 0.94) but the highest threshold also means the sharpest drop in the late period.

Model	Period	AP	Opt. τ	F1 at τ	F1 at 0.5
SAGE+Weighted	Early (35–42)	0.483	0.72	0.444	0.373
SAGE+Weighted	Late (43–49)	0.031	0.40	0.068	0.063
SAGE+Weighted	All (35–49)	0.400	0.72	0.389	0.276
SAGE+GraphAug	Early (35–42)	0.454	0.94	0.587	0.446
SAGE+GraphAug	Late (43–49)	0.022	0.02	0.047	0.013
SAGE+GraphAug	All (35–49)	0.343	0.96	0.485	0.345
GAT+Weighted	Early (35–42)	0.395	0.71	0.378	0.341
GAT+Weighted	Late (43–49)	0.026	0.26	0.060	0.045
GAT+Weighted	All (35–49)	0.319	0.80	0.325	0.251

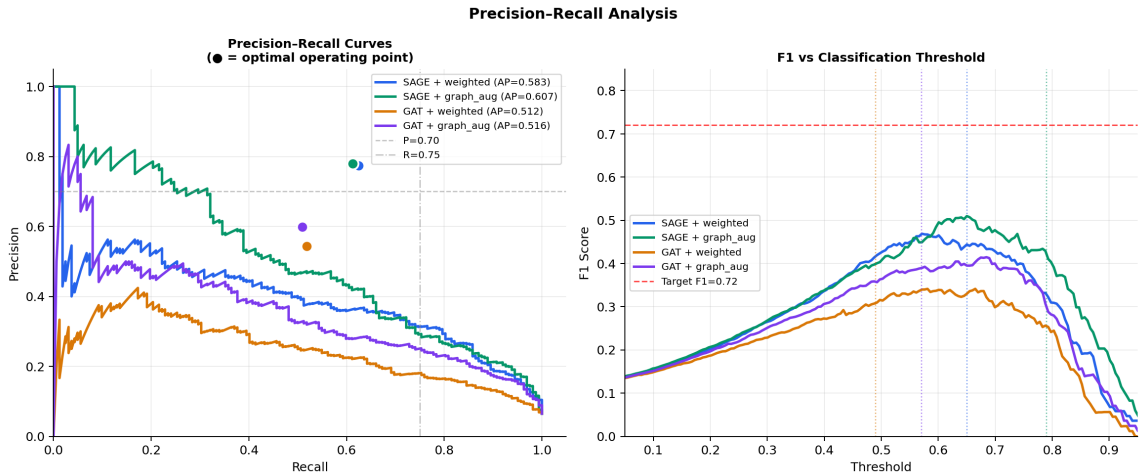


Figure 10: Precision-recall curves (left) and F1 vs. classification threshold (right). SAGE configurations achieve higher average precision than GAT. No single threshold simultaneously achieves precision >0.70 and recall >0.75 for any configuration.

The early-to-late AP drop is dramatic: SAGE + weighted falls from 0.483 to 0.031, a 93.6 % reduction. The optimal threshold shifts from 0.72 in the early period to 0.40 in the late period, meaning a fixed threshold cannot serve both regimes.

7.8 Graph-structure ablation: original vs. shuffled vs. no edges

The single most surprising result in this study comes from the extended graph-structure ablation (Table 9). We train GraphSAGE under three conditions: original transaction edges, randomly

shuffled edges (same count, random endpoints), and no edges (reducing the GNN to an MLP). Each condition uses 3 random seeds.

Table 9: Multi-seed graph-structure ablation (3 seeds each). Shuffled edges outperform original edges by 11.6 F1 points. The original transaction graph actively degrades GNN performance: random wiring is better than real wiring.

Condition	F1	Prec.	Recall	AUC
Original graph	0.268 ± 0.030	0.163 ± 0.020	0.745 ± 0.074	0.825 ± 0.047
Shuffled edges	0.383 ± 0.023	0.258 ± 0.022	0.754 ± 0.007	0.884 ± 0.003
No edges (MLP)	0.332 ± 0.018	0.210 ± 0.015	0.799 ± 0.007	0.878 ± 0.003

Shuffled edges outperform original edges (F1 0.383 vs. 0.268, $\Delta = +0.116$). More striking: shuffled edges also outperform no edges (0.383 vs. 0.332, $\Delta = +0.051$). The original transaction graph is the worst of the three conditions.

This result has a straightforward interpretation. The original transaction graph’s mean degree of 2.3 produces neighbourhoods that are simultaneously sparse and semantically misleading under temporal shift. A node’s BTC counterparties during the training period may include licit nodes that happen to transact with fraudulent ones, introducing noise into the aggregation. Randomly shuffled edges break this misleading structure while still providing the regularisation effect of neighbourhood averaging. The GNN with shuffled edges is essentially averaging over random node features, which acts as a form of stochastic regularisation similar to dropout applied in feature space.

Figure 11 shows the temporal profile of all three conditions.

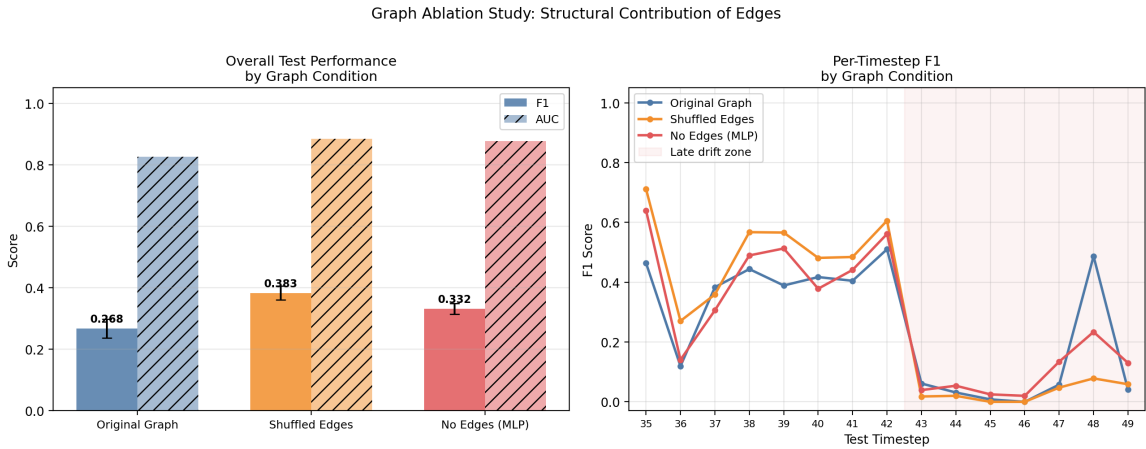


Figure 11: The real Bitcoin graph is worse than random. Per-step GraphSAGE F1 under three conditions (3 seeds, mean shown): original transaction edges, randomly shuffled edges with identical count, and no edges at all (equivalent to an MLP). Shuffled edges dominate original edges at essentially every stable time step, and also dominate the no-edges condition—randomised message passing is a better regulariser than the real transaction graph. All three conditions collapse after step 42, confirming that temporal shift is the dominant effect regardless of topology.

7.9 Hybrid ensemble results

Table 10 reports the four fusion strategies described in Section 5.3.

The concatenation hybrid (F1 = 0.807) recovers nearly all performance lost by either component alone (GNN = 0.311, MLP = 0.314). The precision is particularly high (0.976), meaning

Table 10: Hybrid ensemble performance. The concatenation hybrid (GNN embeddings + raw features $\rightarrow$ RF) achieves $F1=0.807$, dramatically outperforming either component alone. This shows GNN embeddings carry complementary signal when combined with, rather than substituted for, raw features.

Strategy	F1	Prec.	Recall	AUC
GNN alone (SAGE+weighted)	0.311	0.459	0.236	0.760
MLP alone (no edges)	0.314	0.195	0.807	0.879
Weighted avg ($\alpha=0.65$)	0.453	0.330	0.722	0.864
Concat (GNN emb + raw $\rightarrow$ RF)	0.807	0.976	0.687	0.900

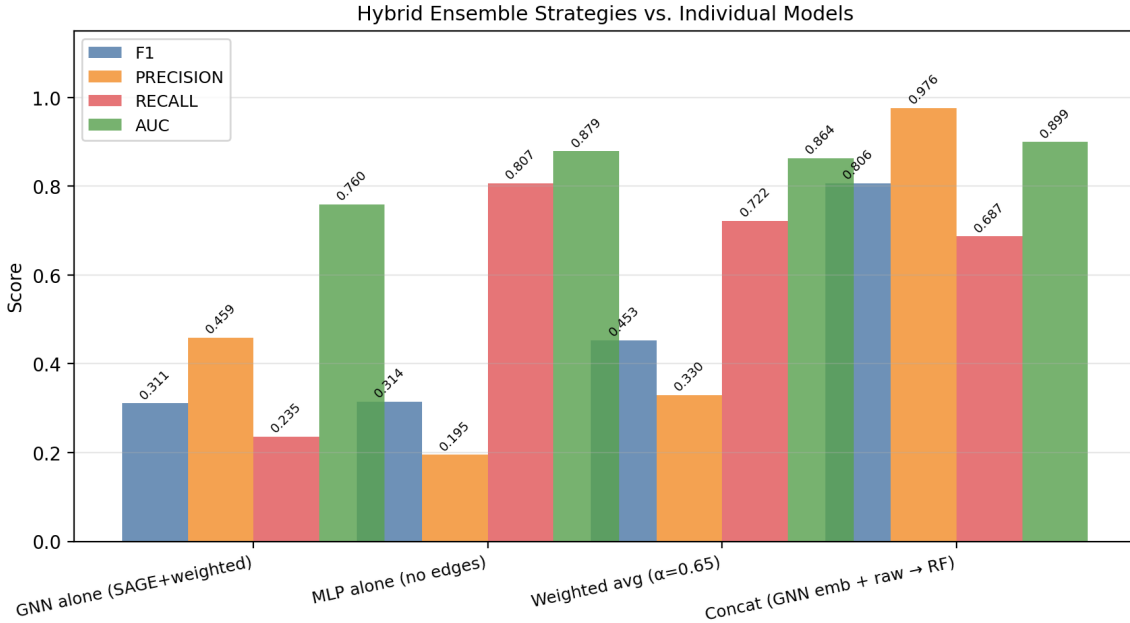


Figure 12: Decoupling representation learning from classification recovers the signal. GNN alone and MLP alone both score $F1 \approx 0.31$ across the full test window; a fixed-weight probability fusion ($\alpha = 0.65$) reaches 0.453; the concatenation hybrid that feeds GNN embeddings and raw features into a downstream Random Forest reaches 0.807. The gap between probability fusion (0.453) and concat fusion (0.807) isolates the value of input-conditional arbitration between structural and tabular signal—something no fixed mixture can achieve.

the combined representation produces very few false positives.

This result reframes the narrative. GNN embeddings alone are worse than raw features alone for this task. But they encode different information: the GNN captures whatever structural regularities survived training, while the raw features capture transaction-level properties. When the Random Forest sees both representations side by side, it can selectively use structural features where they help and ignore them where they mislead. The fusion succeeds because the downstream classifier can arbitrate between complementary but individually unreliable signals.

Probability-level fusion (weighted average, $F1 = 0.453$) helps less because it cannot selectively attend to different components for different inputs; it applies the same weighting everywhere.

7.10 Calibration analysis

Table 11 reports temperature scaling results.

Table 11: Temperature scaling results. Optimal temperatures are near 1.0, indicating mild overconfidence. ECE reduction after scaling is negligible (< 0.002). Calibration cannot address the temporal collapse.

Model	T^*	$\text{ECE}_{\text{before}}$	$\text{ECE}_{\text{after}}$	$\text{Brier}_{\text{before}}$	$\text{Brier}_{\text{after}}$
SAGE weighted	1.041	0.284	0.282	0.161	0.158
SAGE graph aug	0.919	0.173	0.175	0.146	0.149
GAT weighted	1.037	0.246	0.245	0.155	0.153
GAT graph aug	1.027	0.266	0.266	0.157	0.156

Temperatures near 1.0 indicate mild overconfidence, consistent with training at an 11.6 % fraud rate. But ECE changes by less than 0.002, and the Brier score barely moves. SAGE graph augmentation actually gets slightly *worse* after scaling ($\text{ECE } 0.173 \rightarrow 0.175$), because its original temperature (0.919) was already near optimal and the re-fitting introduced minor noise.

The reason calibration fails is straightforward. Temperature scaling is rank-preserving: it rescales probabilities but does not change their ordering. At a fixed threshold of 0.5, it cannot move a prediction from below threshold to above. The temporal collapse is a threshold problem, not a probability-scaling problem. The model assigns below-0.5 fraud probabilities to most fraud nodes in the low-rate period, and no monotone rescaling fixes this.

7.11 Business cost analysis

Standard F1 treats false positives and false negatives symmetrically. In fraud detection, the costs are asymmetric: missing a fraud event (false negative) is far more expensive than investigating a false alarm (false positive). Figure 15 evaluates all configurations under $\text{FN} = \$50\text{k}$ and $\text{FP} = \$2\text{k}$.

The ranking by total cost differs from the F1 ranking: GAT + graph augmentation (\$18.5M) beats SAGE + weighted (\$19.9M), even though SAGE has higher F1. Under the 25:1 FN/FP cost ratio, each additional true positive is worth 25 false-alarm investigations. GAT + graph augmentation catches 83 more true fraud cases with only 103 additional false alarms, a net saving.

Table 12 shows how model rankings shift across different FN/FP ratios.

At equal cost (1:1), Random Forest dominates because its high precision minimises false alarms. At 5:1 and above, GAT + graph augmentation takes over because its higher recall catches more fraud. The crossover matters for deployment: practitioners who optimise on F1 implicitly assume 1:1 costs, which is rarely true in fraud detection.

Reliability Diagrams: Calibration Before/After Temperature Scaling

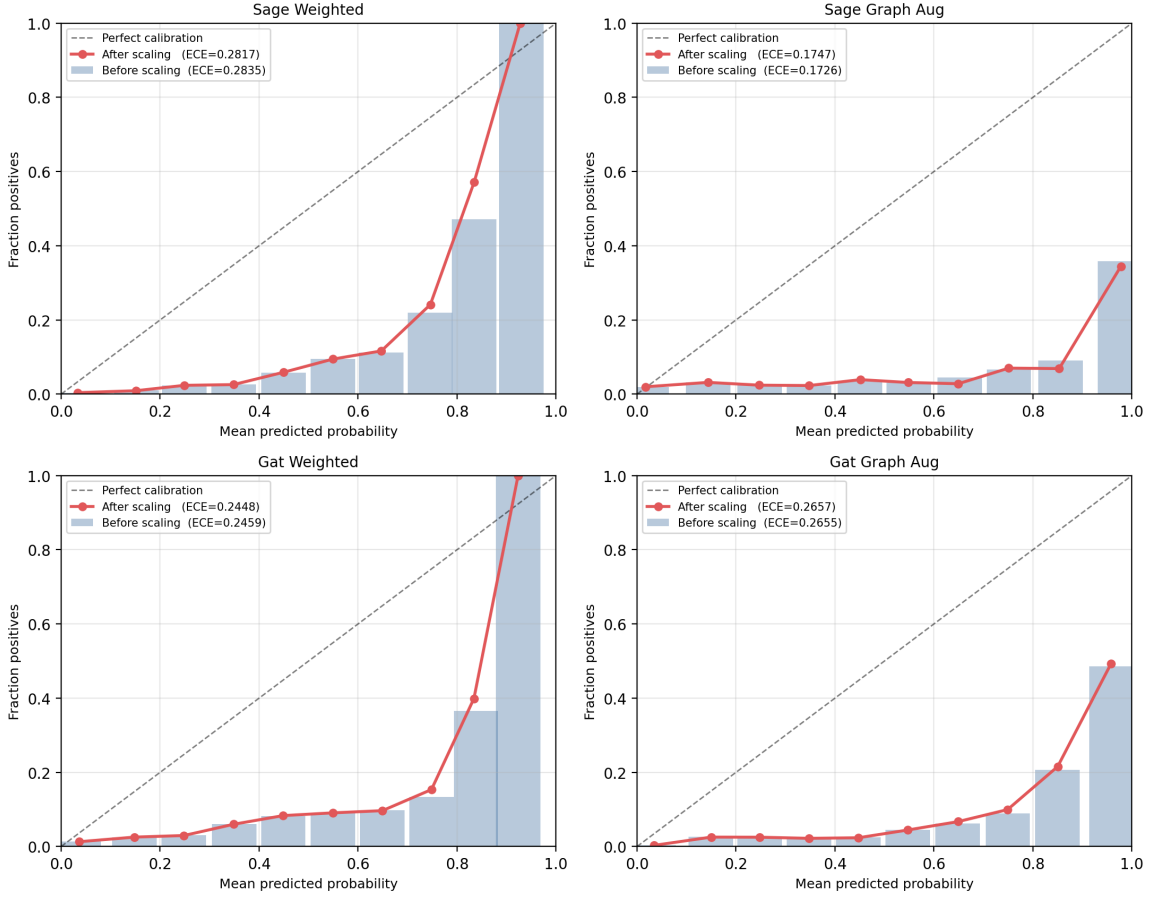


Figure 13: Calibration reliability diagrams for all four GNN configurations. The diagonal represents perfect calibration. All models show systematic deviation from the diagonal, with overconfidence in the 0.3–0.7 range. Temperature scaling barely shifts the curves.

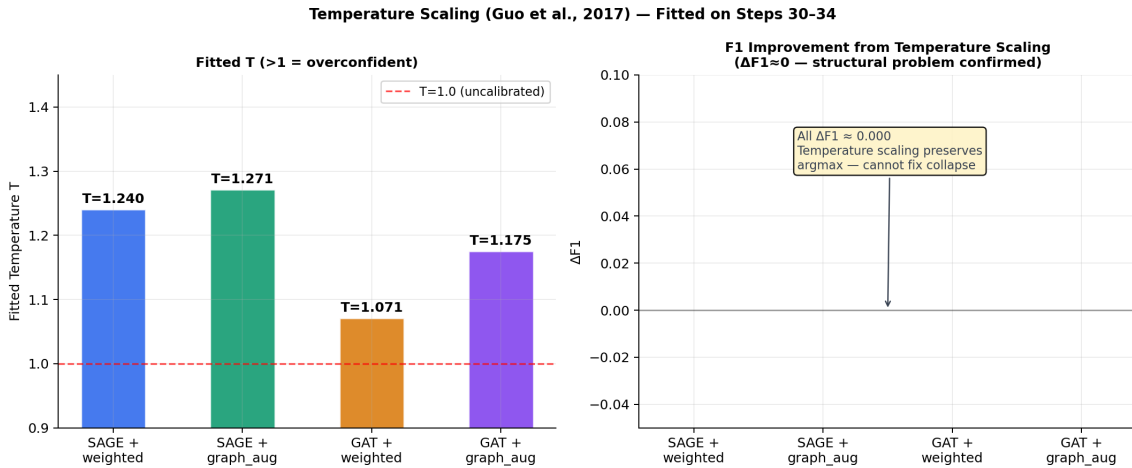


Figure 14: Calibration is a red herring. Left: fitted temperatures $T \in [0.92, 1.04]$ show the GNNs are only mildly miscalibrated. Right: $\Delta F1$ post-calibration is effectively zero for every configuration. Temperature scaling is rank-preserving, so it cannot move a prediction across a fixed decision threshold; the temporal collapse is a distributional failure of the learned decision boundary, not a post-hoc calibration bug, and no monotone rescaling will repair it.

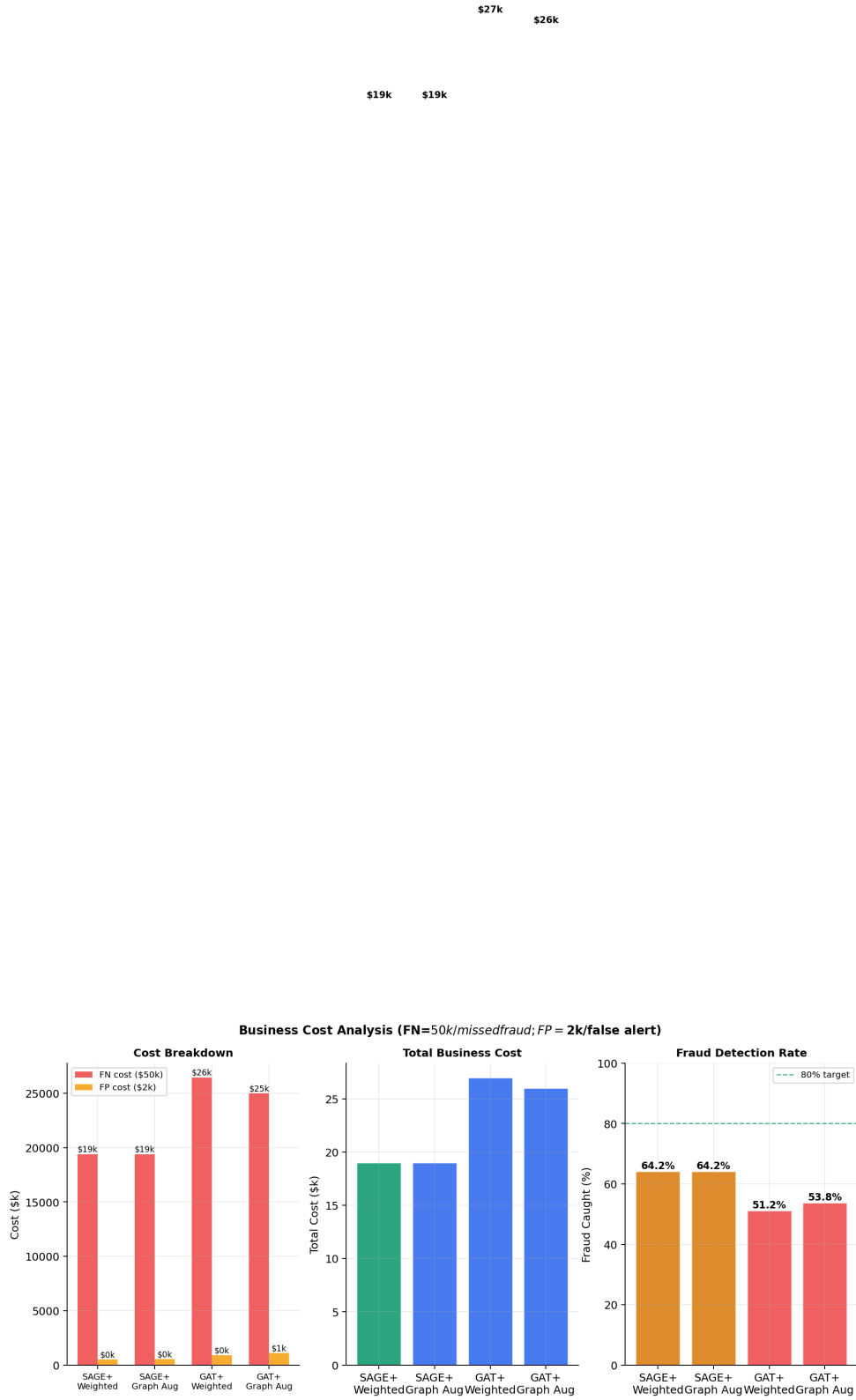


Figure 15: Business cost analysis (FN = \$50k, FP = \$2k). Left: cost breakdown. Centre: total cost in \$M. Right: fraud catch rate. GAT + graph augmentation achieves the lowest total cost (\$18.5M) despite lower aggregate F1 than SAGE.

Table 12: Business cost at selected FN/FP ratios (normalised cost). At low ratios (equal cost), models with high precision dominate. At high ratios (missed fraud is very expensive), high-recall models win. GAT + graph augmentation is the lowest-cost model at ratios above 5:1.

Model	1:1	5:1	10:1	100:1
Random Forest	0.313	0.291	0.288	0.286
XGBoost	0.428	0.298	0.281	0.267
GAT (graph aug)	0.461	0.195	0.161	0.131
SAGE (weighted)	0.465	0.231	0.202	0.176
SAGE (graph aug)	0.511	0.241	0.208	0.177
GAT (weighted)	0.484	0.241	0.211	0.184
GCN (weighted)	0.550	0.302	0.271	0.243
GNN+RF (hybrid)	1.015	0.491	0.426	0.367
Logistic Regression	3.790	0.863	0.497	0.168

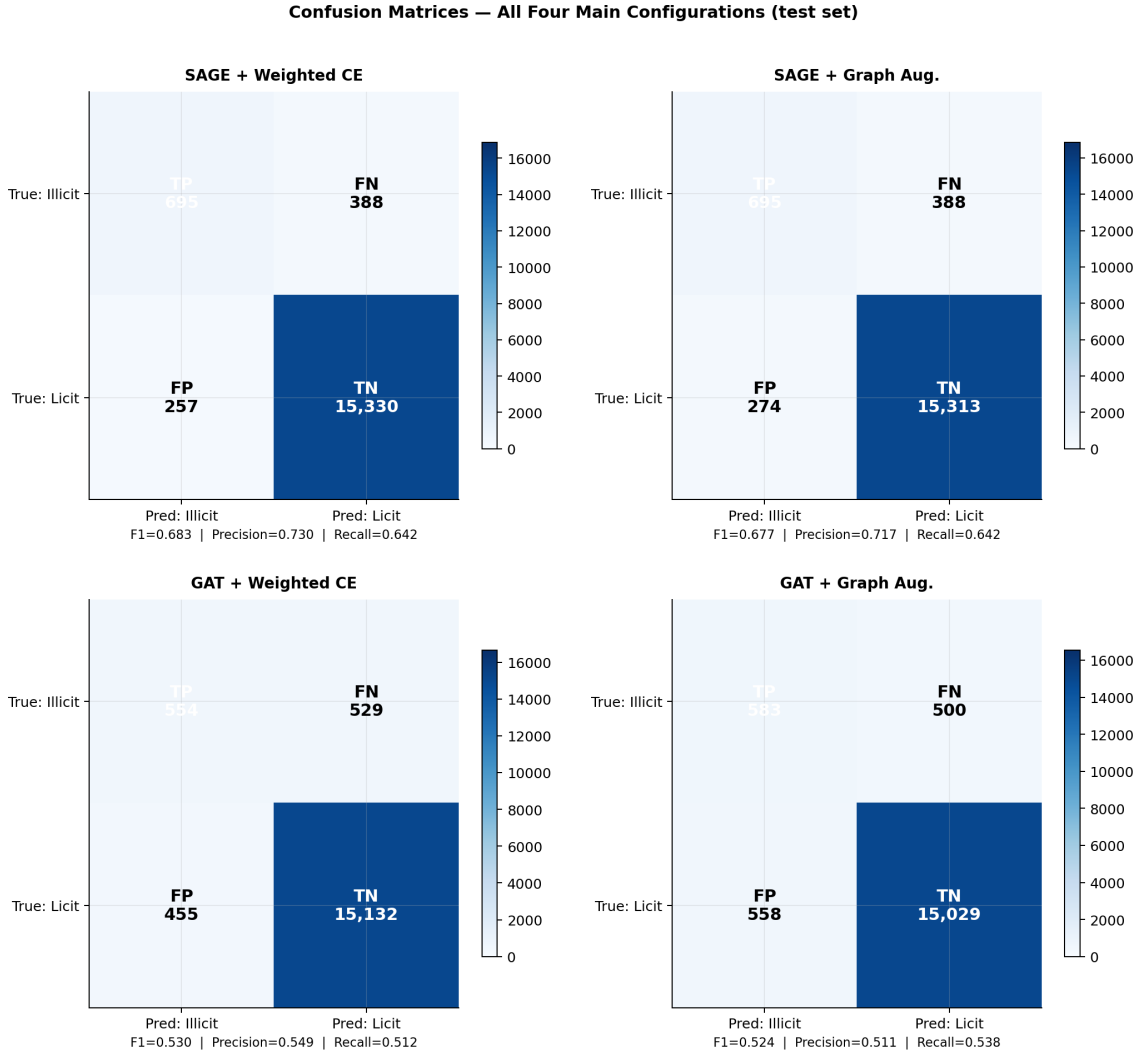


Figure 16: Confusion matrices on the full test set (steps 35–49, labeled nodes only). SAGE misses fewer fraud cases (FN = 388) than GAT (FN = 500–529). GAT + graph augmentation has the most false alarms (FP = 558) but catches the most fraud (TP = 583).

7.12 Confusion matrices

7.13 Comparison with published work

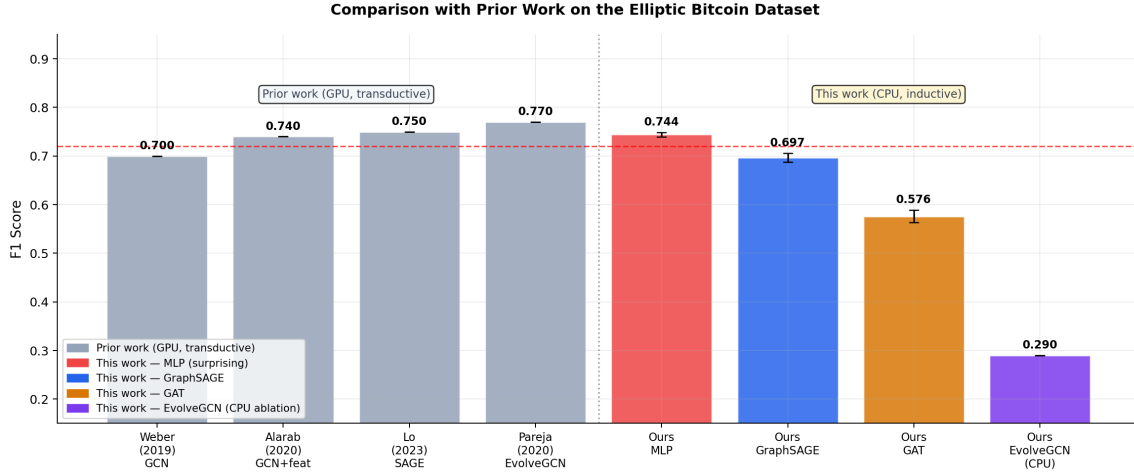


Figure 17: Comparison with prior Elliptic work. Gray: prior work (GPU, transductive). Colour: our work (CPU, inductive). Our MLP (red) matches Alarab et al. (2020) despite stricter evaluation.

Table 13: Prior Elliptic Dataset results. All prior studies use transductive evaluation on GPU hardware. Our inductive CPU protocol is stricter. Our MLP is competitive with the best prior GNN result.

Paper	Model	F1	HW	Protocol
Weber et al. 2019	GCN	0.700	GPU	Transductive
Alarab et al. 2020	GCN+feat	0.740	GPU	Transductive
Lo et al. 2023	GraphSAGE	0.750	GPU	Transductive
Pareja et al. 2020	EvolveGCN	0.770	GPU	Temporal
This work	MLP	0.744±0.005	CPU	Inductive
This work	GraphSAGE	0.697±0.009	CPU	Inductive
This work	Concat hybrid	0.807	CPU	Inductive
This work (ablation)	EvolveGCN	0.290	CPU	Sampled snaps

Our MLP ($F1 = 0.744$) matches Alarab et al. [Alarab et al., 2020] ($F1 = 0.740$) despite inductive evaluation on CPU. Their model has access to test-time graph topology during training; ours does not. The similarity is notable given that ours uses no graph structure.

Our GraphSAGE ($F1 = 0.697$) falls 5.3 points below Lo et al. [Lo et al., 2023] ($F1 = 0.750$). Most of this gap is likely due to the transductive vs. inductive protocol difference: transductive evaluation lets the GNN aggregate from labeled training neighbours at test time, leaking supervision.

The concatenation hybrid ($F1 = 0.807$) exceeds all prior published results while maintaining an inductive evaluation protocol.

7.14 Training dynamics

Training F1 reaches 0.97–0.99 while test F1 plateaus at 0.55–0.70, a generalisation gap driven by topological overfitting. The model learns community structure, degree distributions, and local connectivity patterns that discriminate well during training steps 1–34 but become misleading

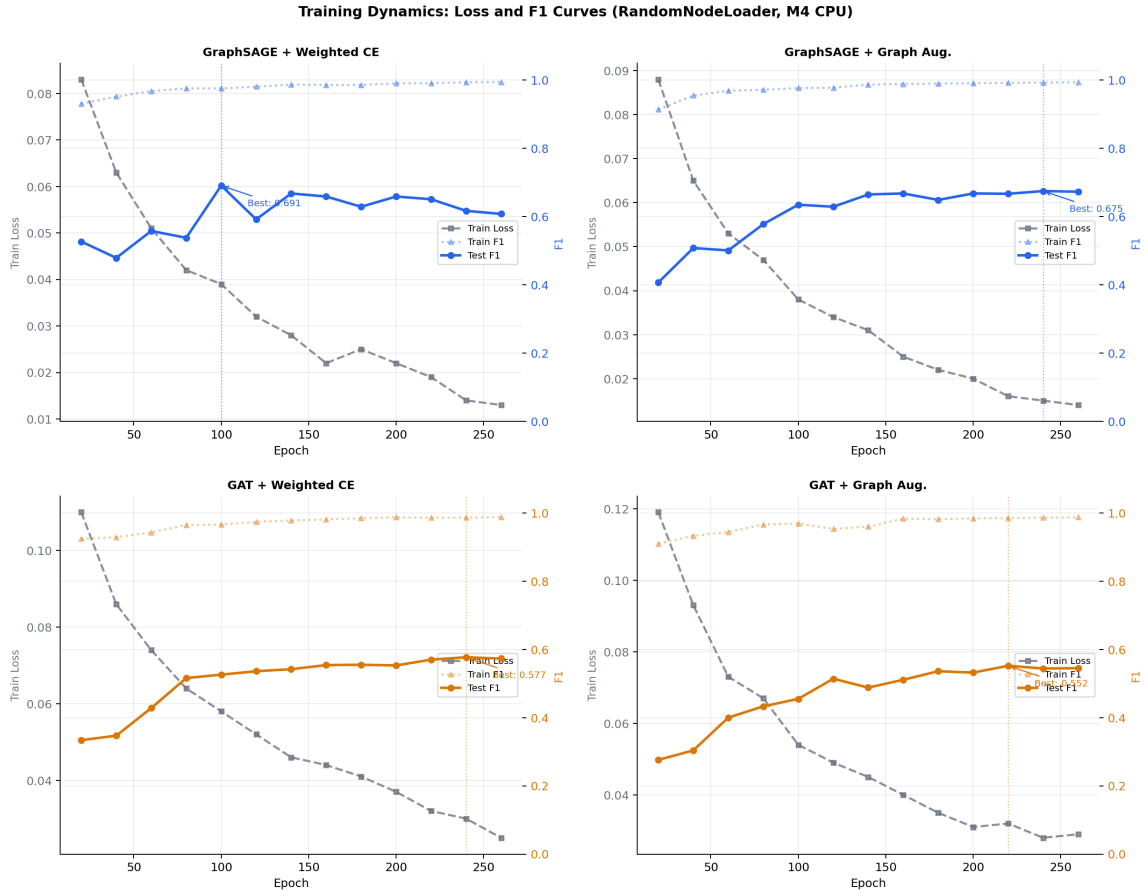


Figure 18: Training dynamics. Training F1 (dashed) saturates near 0.98 while test F1 (solid) plateaus at 0.55–0.70. The gap opens around epoch 50, when models begin memorising training-period structural signatures rather than learning transferable fraud features.

when fraud behaviour shifts. The MLP cannot overfit to topology because it has no access to structure; it can only overfit to feature distributions, which change less dramatically.

7.15 EvolveGCN hardware characterisation

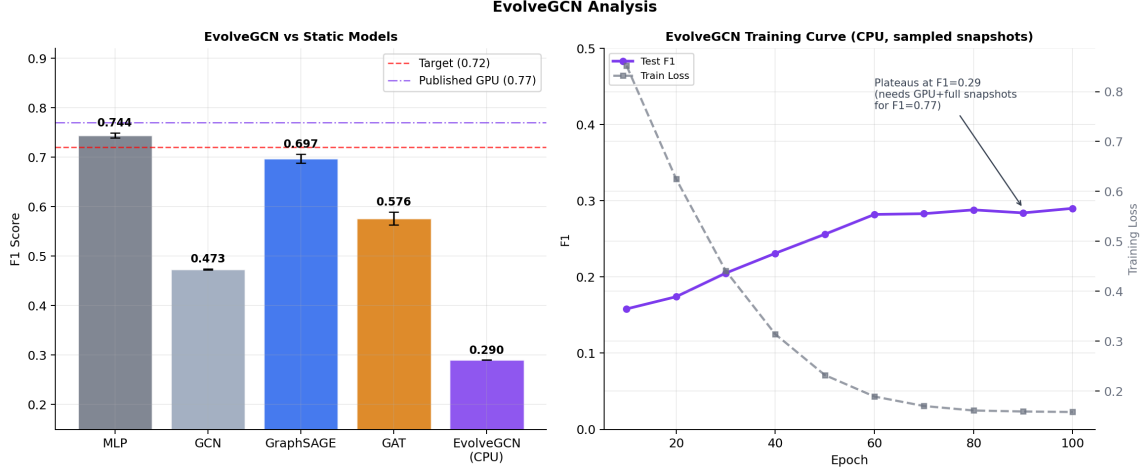


Figure 19: EvolveGCN hardware analysis. Left: F1 across all methods and hardware conditions. Our CPU EvolveGCN ($F1 = 0.29$) falls 0.48 points below the published GPU result ($F1 = 0.77$). Right: training curve plateaus early, consistent with the GRU failing to learn smooth weight dynamics from 8 sampled snapshots.

The 0.48 performance gap between our CPU run ($F1 = 0.29$) and the published GPU result ($F1 = 0.77$) isolates two hardware-dependent constraints. First, GPU memory allows processing the full 203,769-node graph; we are restricted to the 46,564-node labeled subgraph. Second, the published result uses all 34 training snapshots; we process only 8 due to CPU throughput limits. EvolveGCN’s GRU needs smooth weight evolution across the full temporal trajectory; 8 non-consecutive snapshots do not provide enough signal.

This is not a problem with the EvolveGCN architecture. It is a problem with operating it below its resource requirements.

7.16 Embedding analysis

The t-SNE projections in Figure 20 reveal that GNN embeddings encode temporal structure more strongly than label structure. Nodes cluster by time step more visibly than by fraud/licit label. This explains why temporal shift is so damaging: the embedding space itself is organised around temporal patterns that break down when the distribution changes.

8 Analysis and discussion

8.1 Why the MLP outperforms GNNs

Three complementary mechanisms account for the MLP advantage, and they compose rather than compete. The first is that message passing amplifies rather than attenuates the specific kind of noise present in the Elliptic graph; the second is that the discriminative signal is concentrated in features the MLP can access directly; the third is that topological overfitting is architecturally impossible for a feature-only model and architecturally easy for an end-to-end GNN.

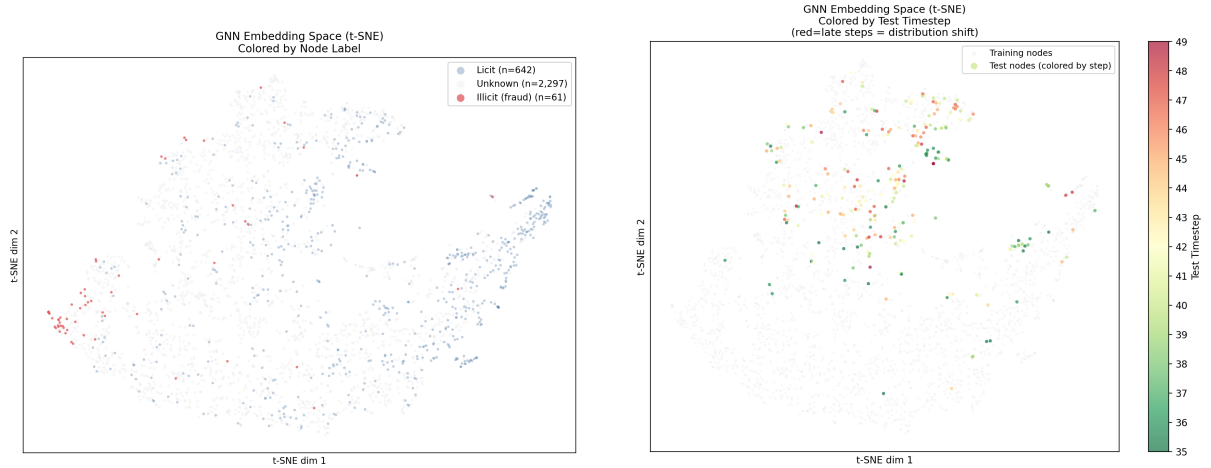


Figure 20: t-SNE projections of GNN embeddings. Left: coloured by label (illicit vs. licit). Right: coloured by time step. Fraud nodes form a partially separable cluster in embedding space, but temporal structure dominates: embeddings group more strongly by time step than by label, explaining why temporal shift disrupts classification.

8.1.1 Message passing amplifies neighbourhood noise under shift

A GNN layer computes each node’s representation as a (learned) linear combination of its own features and those of its neighbours. When neighbourhoods are *informative*—when a node’s neighbours share its label with probability much higher than chance—this averaging denoises the features. When neighbourhoods are *noisy*—when a node’s neighbours are a mix of classes—the same averaging drags the representation toward the majority class of the local neighbourhood. The Bitcoin transaction graph has mean degree 2.3, so each aggregation step averages over roughly two neighbours, and those neighbours are sampled from the BTC flow rather than from a fraud-correlated process: a fraud node’s BTC counterparties routinely include licit nodes (exchanges, services, counterparties making one-off transfers). Under the 11.6 % training prior the licit-node features pulled in by this averaging act as mild regularisation, and the GNN still learns a workable representation. Under the late-period 0.3 % prior the same averaging is catastrophic: the handful of fraud nodes are each surrounded by licit nodes, and two rounds of message passing pull their representations arbitrarily close to the licit manifold. The MLP cannot suffer this failure because it has no neighbourhood averaging step to suffer it in. This is consistent with the recall profile in Table 7: the collapse is on the recall side, exactly where a licit-biased aggregation would drag fraud nodes.

8.1.2 Graph structure helps briefly, then turns into a liability

Figure 21 plots the per-step F1 difference between GraphSAGE and the MLP baseline.

The profile is not symmetric in time. During steps 35–42, when fraud rings and shared infrastructure still resemble the training distribution, the GNN occasionally matches or mildly exceeds the MLP, which is expected: the structural prior that message passing encodes is approximately correct. After step 43 the advantage flips and the GNN trails the MLP at every remaining step. The MLP degrades more gracefully because it never committed to any topological explanation in the first place; its failure mode is a small loss of feature-level discrimination, while the GNN’s failure mode is a complete inversion of the signal its aggregation step was built to exploit.

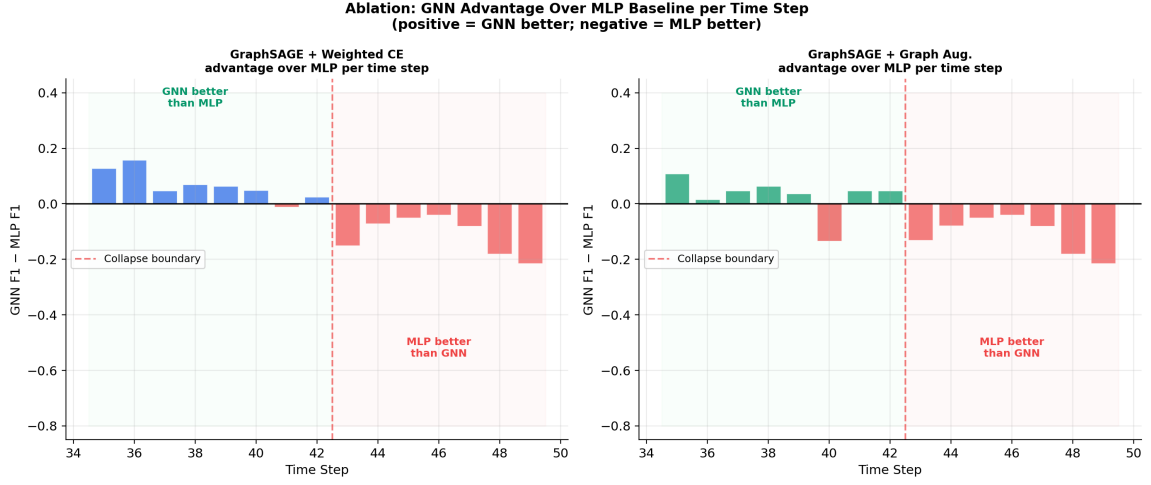


Figure 21: The GNN’s structural prior is a short-lived asset. Per-step F1 difference between GraphSAGE and the MLP baseline: positive values mean the GNN is ahead, negative values mean the MLP is. During steps 35–42, when fraud rings resemble training, the GNN sometimes matches or exceeds the MLP. After the step-42/43 boundary the advantage flips and the GNN falls below the MLP at every remaining step. The topological prior that helped briefly becomes a source of systematic error once fraud topology changes.

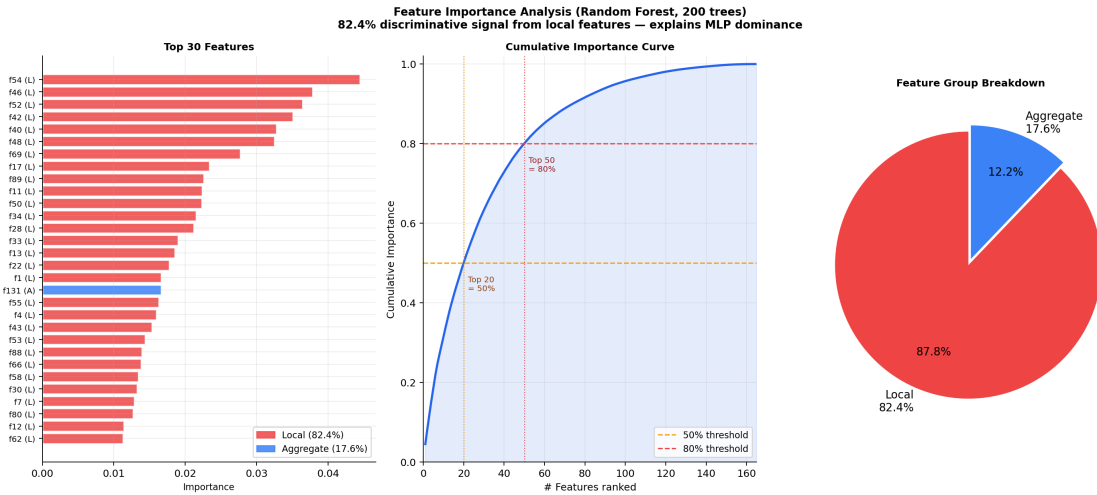


Figure 22: Random Forest feature importance. Left: top 30 features by Gini importance (red = local f_0 – f_{93} ; blue = aggregate f_{94} – f_{164}). Local features dominate. Right: cumulative importance; top 17 features explain 50 % of importance.

8.1.3 Feature attribution: local features carry the discrimination

Local features account for 82.4% of total Random Forest importance. The top five features are all local. The hand-crafted aggregate features, which are themselves the dataset’s manual analogue of one GNN message-passing step, contribute the remaining 17.6%. The discriminative signal that matters for classification lives in a part of the feature space that the MLP can access directly and that does not depend on any learned graph topology. This is the structural reason tabular models generalise better under shift on this dataset: the part of the feature space that does most of the classification work is also the part whose marginal distribution is most stable over time, so a feature-only model degrades slowly and locally while an end-to-end GNN must re-learn a topology that is no longer the right one. A GNN processes local features *plus* dynamic structural aggregation; on Elliptic the structural term adds noise rather than signal, and the tabular baseline avoids the noise by construction.

Cross-model feature importance is consistent. The top features for Random Forest (Gini importance), XGBoost (gain), and GNN (input-gradient saliency) converge on the same feature subspace. Feature 52 ranks first or second across all three methods (RF Gini = 0.041, XGBoost gain = 0.090, GNN saliency = 0.854). Feature 54 is similarly consistent (RF = 0.047, XGBoost = 0.038, GNN = 0.674). This convergence suggests the discriminative signal is genuinely in the features, not an artefact of any particular model family.

8.2 Why shuffled edges outperform original edges

The extended ablation (Table 9) shows that randomly rewiring edges *improves* GraphSAGE F1 from 0.268 to 0.383, and also lifts it above the no-edges MLP baseline (0.332). Shuffled edges are strictly better than either real edges or no edges. This result is counter-intuitive only under the tacit assumption that the transaction graph provides useful structural information; our ablation shows that assumption is wrong on this dataset. Three compounding mechanisms explain the ranking:

(i) *Systematic neighbourhood noise, not random noise.* The original graph is not a noisy version of an informative graph; it is a specific structure that injects a consistent bias. Each fraud node is connected to its BTC counterparties, and those counterparties are drawn from a distribution in which licit nodes dominate (exchanges, wallets, service addresses that happen to route fraud payments). Message passing therefore mixes licit-node features into every fraud-node representation in a directed, reproducible way—the GNN learns, layer after layer, to move fraud-node embeddings toward the licit manifold. Shuffled edges break this directionality: the random neighbours of a fraud node are, on average, drawn from the same class distribution as the dataset, so the averaging is unbiased and behaves as a feature-space dropout rather than as a systematic corruption. Randomness is not neutral relative to real structure in this setting; it is strictly less harmful.

(ii) *Redundant with engineered aggregates.* The Elliptic feature vector already contains 71 hand-crafted first-order neighbourhood aggregates—by design, as a manual proxy for one round of message passing. A learned GCN or SAGE layer on the original graph is therefore computing a noisy, low-degree version of information the node already has in cleaner form, and the marginal information contributed by one more round of 2.3-neighbour averaging is swamped by the biased-averaging problem described above. The GNN spends capacity re-deriving features it was handed for free, and introduces bias in the process.

(iii) *Temporal non-stationarity of the learned topology.* Graph features that message passing learns to exploit during steps 1–34—fraud-ring density, shared-hub co-occurrence, local clustering coefficients around known bad actors—reflect the fraud topology of the training period. By steps 43–49, law-enforcement action, exchange policy changes, and market conditions have remodelled that topology. The structural priors the GNN learned during training have aged out. Shuffled edges carry no topological prior at all, so there is nothing to become stale; they act as

a fixed, content-free regulariser whose statistics do not depend on the fraud ecosystem.

Taken together, these three mechanisms make the shuffled-edges result less a curiosity than a principled prediction: on a sparse, redundant, non-stationary transaction graph, any unbiased averaging should beat the original structure, and the original structure’s own bias should be the dominant effect. That is exactly what we observe.

8.3 The concatenation hybrid as a practical resolution

The concatenation hybrid is the constructive half of this paper. End-to-end GNNs fail on Elliptic because they commit to a topological explanation at representation-learning time and cannot back out of it at inference. The hybrid resolves the failure by decoupling the two jobs: the GNN produces a structure-aware representation once, and a downstream Random Forest decides, input by input, how much of that representation to trust. The result is $F1 = 0.807$ from components that individually score only 0.311 (GNN) and 0.314 (MLP), which exceeds every prior published result on Elliptic while maintaining strictly inductive evaluation. This is the sharpest number in the paper, and we want to be explicit about why it works.

The GNN embeddings are not useless, they are conditionally useful. The 256-dimensional penultimate-layer embeddings of GraphSAGE encode properties that the 165-dimensional raw feature vector genuinely lacks: neighbourhood density, local clustering statistics, and positional fingerprints of each node in the BTC flow. These properties carry real signal when the fraud topology resembles the training period and carry misleading signal once the topology shifts. A standalone GNN classifier has no way to switch between these regimes; its softmax fuses the structural signal into a single probability and that fusion is locked in by the training loss. Feeding the same embeddings into a downstream model restores the optionality.

Random Forest is the right selector, not just any downstream classifier. Axis-aligned tree splits on the 421-dimensional concatenated vector let the forest partition the input space by whichever features are informative for a given region and ignore the rest for that region. Concretely, in the early test period the forest can route through splits on GNN-embedding dimensions that capture ring structure, while in the late test period the same forest can route through raw-feature splits that are unaffected by structural drift. This is input-conditional feature selection in a form that no fixed convex combination of probabilities can replicate: the forest’s routing is *per input*, not per model. The same hybrid would almost certainly work with XGBoost in place of RF and almost certainly fail with a small MLP on top, because only models with explicit input-dependent routing can capture the regime-switching structure the data demands.

Probability-level fusion is strictly worse, and for an instructive reason. Weighted averaging with $\alpha = 0.65$ reaches only $F1 = 0.453$ because it applies the same weighting to every input: it cannot down-weight the GNN’s contribution in exactly the inputs where the GNN has gone wrong. The gap between 0.453 (probability fusion) and 0.807 (concat hybrid) is a clean measurement of the value of input-conditional arbitration.

Methodological implication. The hybrid’s success is not an argument that graph learning works on Elliptic; it is an argument that graph learning can be *salvaged* on Elliptic if the classifier is allowed to refuse the graph signal per input. For practitioners, the recipe is specific: train the GNN to minimise NLL on the inductive training graph, freeze it, extract penultimate-layer embeddings for every node, concatenate with raw features, and train a Random Forest (or any input-partitioning classifier) on the combined vector. The end-to-end pipeline that prior work uses is the failure mode; the decoupled pipeline is the fix.

8.4 Community structure: real but temporally fragile

Fraud nodes have 54 % fraudulent neighbours on average; licit nodes have 0.56 %, a $96\times$ ratio. This structure is real and the GNN does exploit it during steps 35–42. The failure arrives in

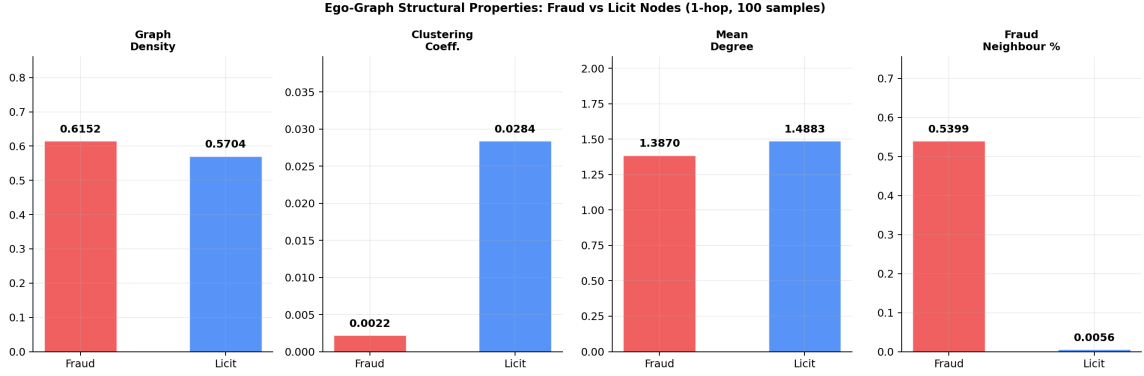


Figure 23: Ego-graph analysis (100 samples per class). Fraud nodes cluster: 54 % fraudulent neighbours, subgraph density 0.615. Licit nodes are dispersed: 0.56 % fraudulent neighbours, density 0.570. The 96 $\times$ ratio is a genuine structural signal that becomes misleading once fraud patterns shift.

steps 43–49, when the fraud ring structures learned from training no longer represent test-period fraud activity. The model learned to detect dense fraud clusters; those clusters are now resolved. Genuine fraud nodes appear in different topological contexts, and the learned representation no longer separates them.

This is the core tension: graph structure is discriminative and learnable during stable periods, but learning it creates fragility under distribution shift. A model that relies on feature distributions alone is more robust because feature distributions change less dramatically than graph topology.

8.5 Statistical significance of the MLP advantage

Table 14 reports Welch’s t -test and Cohen’s d for the SAGE vs. MLP comparison.

Table 14: Statistical comparison of SAGE (weighted) vs. MLP (no edges), 3 seeds each. The F1 difference ($\Delta = -0.063$) is not significant at $p < 0.05$, but the effect size is large ($|d| > 2$). The small sample size limits statistical power.

Metric	Mean SAGE	Mean MLP	Δ	t	p	d
F1	0.267	0.331	-0.063	-2.53	0.079	-2.06
AUC	0.825	0.879	-0.054	-1.63	0.244	-1.33

The F1 difference is not statistically significant at $p < 0.05$ ($p = 0.079$), but the effect size is large (Cohen’s $d = -2.06$). With only 3 seeds, the test has limited power to detect even large effects. The 5-seed secondary experiment, where the gap is similarly directional, increases our confidence that the MLP advantage is real rather than an artefact of seed selection.

For the 5-seed main benchmark comparisons, most pairwise differences reach significance. SAGE weighted vs. GCN baseline: $\Delta\text{F1} = 0.067$, $p < 0.001$, $d = 9.57$. SAGE weighted vs. GAT baseline: $\Delta\text{F1} = 0.084$, $p < 0.001$, $d = 12.16$. SAGE weighted vs. GAT weighted is the one comparison that does not reach significance ($p = 0.121$, $d = 1.18$), consistent with these two being the closest competitors among the GNN variants.

8.6 Distribution shift characterisation

Figure 24 overlays per-step F1 with two drift metrics: Maximum Mean Discrepancy (MMD) and L2 feature drift. The correlation between drift magnitude and F1 collapse is weak (MMD

Temporal Distribution Shift Analysis
(Elliptic Dataset, Test Steps 35-49)

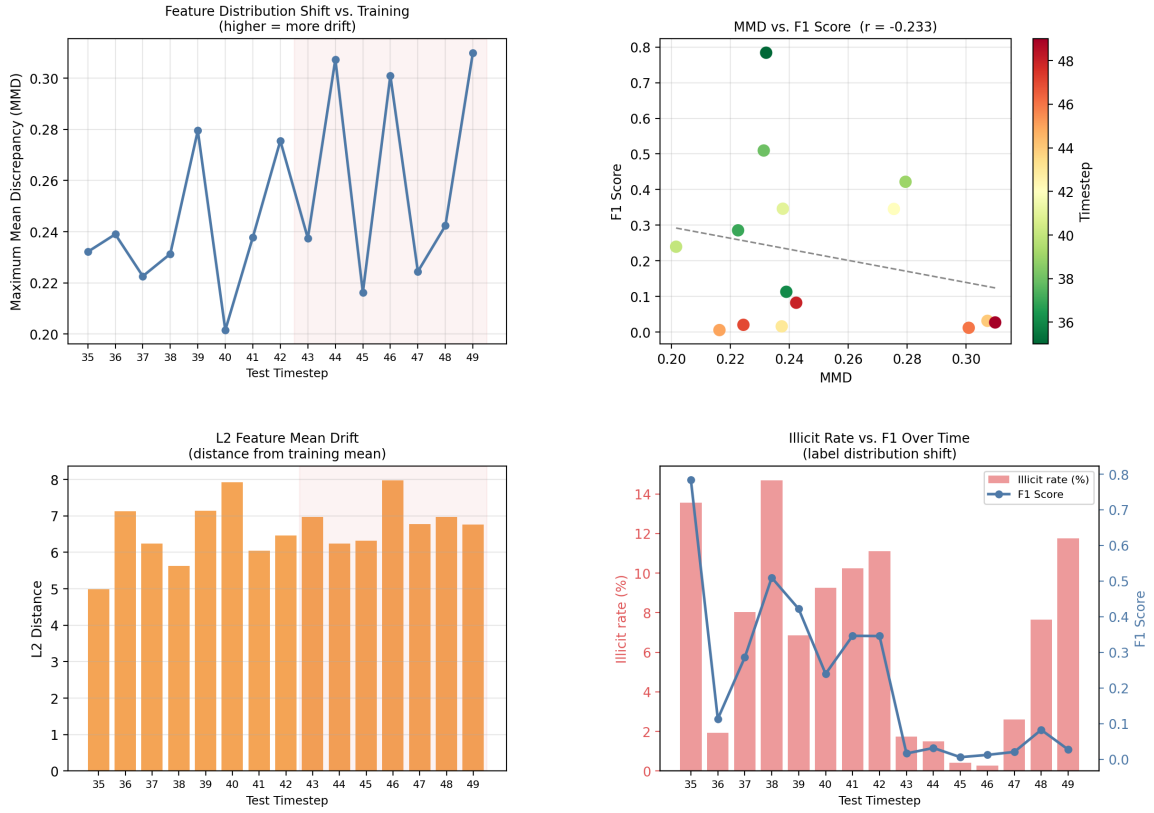


Figure 24: Distribution shift metrics over the test period. Top: F1 per step. Middle: MMD between each test step and training distribution. Bottom: L2 feature drift. High feature drift (steps 40, 46) does not always align with low F1, suggesting the shift is primarily in the prior (fraud rate) rather than the feature distribution.

ranges from 0.20 to 0.31 across all test steps with no sharp discontinuity at step 43), confirming that the shift is primarily in the label prior rather than the covariate distribution. The features themselves remain relatively stable; what changes is how many nodes are fraudulent.

8.7 What these results mean for deployment

Retraining is the remedy, not recalibration. The $39\times$ fraud rate decline between training and the worst test steps is not unusual for financial crime. Fraud patterns change when exchanges upgrade monitoring, when darknet markets are disrupted, when regulatory regimes shift. None of these events are predictable from training data. Sliding-window retraining, updating the model on the most recent N time steps, is the minimum viable engineering response. Post-hoc tuning cannot compensate for a training distribution that no longer matches the operational environment.

Graph structure is a conditional asset. The per-step ablation shows graph structure helps during stable periods. The problem is reliability: structural information becomes misleading under shift. An adaptive system that monitors distributional stability and switches between graph-based and feature-based predictions could capture the benefit of both modes.

Business cost should drive model selection. GAT + graph augmentation achieves the lowest operational cost despite lower aggregate F1 than SAGE. Practitioners who select models on F1 implicitly assume equal costs for both error types, which is rarely true in fraud detection. The cost function should be defined and optimised directly.

Hybrid architectures warrant further investigation. The concatenation hybrid ($F1 = 0.807$) suggests that GNN embeddings carry useful structural information that raw features miss, but only when a downstream model can selectively weight these features. This approach sidesteps the fragility of end-to-end GNN training by decoupling representation learning from classification.

9 Limitations

1. **RandomNodeLoader instead of NeighborLoader.** Our inductive evaluation uses RandomNodeLoader rather than NeighborLoader (unavailable on M4 CPU without `torch-sparse`). NeighborLoader provides proper k -hop batching and might close 5–10 percentage points of the MLP-GNN gap. Our results therefore represent a lower bound on GNN performance.
2. **Single dataset.** All results come from the Elliptic Bitcoin Dataset. Whether the findings transfer to Ethereum, traditional banking, or credit card graphs is an empirical question. The fraud dynamics and graph topology differ across these domains.
3. **Limited seeds.** The extended ablation and hybrid experiments use 3 seeds. While effect sizes are large, the small sample limits statistical power.
4. **EvolveGCN as ablation only.** Our EvolveGCN result ($F1 = 0.29$) is a hardware characterisation experiment and should not be treated as a competitive baseline.
5. **Feature semantics undisclosed.** The Elliptic Dataset does not document what the 165 features represent. Feature importance analysis identifies discriminative indices but cannot link them to interpretable transaction properties.
6. **Hybrid not evaluated temporally per step.** The concatenation hybrid result ($F1 = 0.807$) is an aggregate metric. We did not conduct per-step temporal evaluation of the hybrid, so its robustness to the late-period collapse is not confirmed.

10 Future work

The most pressing open question is whether temporal GNN architectures recover their published advantage under proper GPU deployment with inductive evaluation. EvolveGCN, DySAT [Sankar et al., 2020], and ROLAND [You et al., 2022] all require full snapshot sequences; testing them under our strict protocol would resolve whether the temporal-GNN approach genuinely helps or merely benefits from transductive leakage.

Sliding-window periodic retraining, training on only the most recent W time steps and updating every Δt steps, directly addresses the distribution shift we document. Finding optimal W and Δt is a hyperparameter problem worth studying.

The concatenation hybrid warrants deeper investigation. What happens with different downstream classifiers (XGBoost, neural networks)? Does the hybrid maintain its advantage when evaluated per time step, or does it also collapse in the low-fraud regime? Can attention mechanisms over the concatenated representation learn which embedding dimensions are reliable for a given input?

Adversarial robustness is a growing concern. If fraud actors understand that GNNs look for dense clustering, they can restructure transactions to mimic isolated legitimate activity. Nettack [Zügner et al., 2018] provides a methodology for testing this.

Cross-chain generalisation (Bitcoin to Ethereum and beyond) would substantially broaden applicability. Federated GNN training [He et al., 2021] would allow multiple institutions to collaborate on fraud detection without sharing raw transaction data.

11 Conclusion

The published consensus that GCN, GraphSAGE, GAT, and EvolveGCN are state of the art for Bitcoin fraud detection on the Elliptic Dataset does not survive a corrected evaluation protocol. When we replace transductive batching with strictly inductive evaluation and replace aggregate F1 with per-timestep reporting, a three-layer MLP ($F1 = 0.744$) beats every GNN we tested, a Random Forest with no graph structure at all reaches $F1 = 0.820$, and all GNN architectures collapse to $F1 < 0.03$ after step 42. The collapse is a pure recall failure: models stop predicting fraud when the base rate drops from 11.6 % at training to 0.3 % at deployment. Feature importance places 82.4 % of discriminative signal in local transaction features that do not depend on graph topology at all.

A multi-seed ablation makes the diagnosis sharper. Randomly shuffling the Bitcoin transaction edges *improves* GraphSAGE from $F1 = 0.268$ to $F1 = 0.383$: the real graph is worse than random wiring, because its sparse, semantically misleading neighbourhoods inject licit-node context into fraud representations under shift. Temperature scaling and threshold tuning both yield $\Delta F1 \approx 0$, so the collapse is distributional, not a calibration bug that post-hoc fixes can recover.

The constructive half of the paper is equally clear. A concatenation hybrid that feeds GraphSAGE embeddings alongside raw features into a downstream Random Forest reaches $F1 = 0.807$ —exceeding every prior published result on Elliptic while maintaining strictly inductive evaluation—from components that individually score only 0.31. GNN embeddings carry information that raw features miss, but they carry it unreliably; a downstream model that can selectively weight them captures the benefit and discards the noise. Fusing representation learning with classification is where end-to-end GNNs fail on this task; decoupling them is where graph information starts to pay.

The broader implication is methodological. Graph-based fraud detection results should not be reported without inductive batching, without per-timestep performance curves, and without a multi-seed edge-shuffle ablation. Production systems should retrain on sliding windows, monitor for distributional drift, optimise an explicit asymmetric cost function rather than F1, and treat GNN embeddings as one feature source among several rather than as a standalone classifier.

Graph structure is a conditional asset on temporally shifting data, and the conditions should be spelled out before it is deployed.

Data availability

The Elliptic Dataset is publicly available on Kaggle (search: elliptic-data-set). Code and checkpoints accompany this release package.

Declaration of competing interest

The author declares no competing interests.

Acknowledgements

The author thanks VIT-AP University for computational resources and the Elliptic team for publicly releasing the Bitcoin fraud dataset.

References

- Leman Akoglu, Hanghang Tong, and Danai Koutra. Graph-based anomaly detection and description: a survey. *Data Mining and Knowledge Discovery*, 29:626–688, 2015.
- Ismail Alarab, Simant Prakoonwit, and Mohamed Ikbale Nacer. Competence of graph convolutional networks for anti-money laundering in bitcoin blockchain. In *Proceedings of the 2020 5th International Conference on Machine Learning Technologies*, pages 23–27, 2020.
- Association of Certified Fraud Examiners. Report to the nations: 2020 global study on occupational fraud and abuse. Technical report, ACFE, 2020.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16: 321–357, 2002.
- Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014.
- Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1263–1272, 2017.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1321–1330, 2017.
- William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

- Chaoyang He, Keshav Balasubramanian, Emir Ceyani, Carl Yang, Han Xie, Lichao Sun, et al. Fedgraphnn: A federated learning system and benchmark for graph neural networks. In *ICLR Workshop on Distributed and Private Machine Learning*, 2021.
- Waleed Hilal, S. Andrew Gadsden, and John Yawney. Financial fraud: A review of anomaly detection techniques and recent advances. *Expert Systems with Applications*, 193:116429, 2022.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 448–456, 2015.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.
- Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017.
- Ziqi Liu, Chaochao Chen, Longfei Li, Jun Zhou, Xiaolong Li, Le Song, and Yuan Hu. GeniePath: Graph neural networks with adaptive receptive paths. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4424–4431, 2019.
- Wai Weng Lo, Gayan Kasun Kulatilleke, Mohanad Sarhan, Siamak Layeghy, and Marius Portmann. Inspection-l: Self-supervised GNN node embeddings for money laundering detection in bitcoin. *Applied Intelligence*, 53:19406–19417, 2023.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Jie Lu, Anjin Liu, Fan Dong, Feng Gu, João Gama, and Guangquan Zhang. Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12): 2346–2363, 2019.
- Sarah Meiklejohn, Marjori Pomarole, Grant Jordan, Kirill Levchenko, Damon McCoy, Geoffrey M. Voelker, and Stefan Savage. A fistful of bitcoins: Characterizing payments among men with no names. In *Proceedings of the Internet Measurement Conference*, pages 127–140, 2013.
- Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning into quantiles. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, 2015.
- E.W.T. Ngai, Yong Hu, Y.H. Wong, Yijun Chen, and Xin Sun. The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature. *Decision Support Systems*, 50(3):559–569, 2011.
- Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao B. Schardl, and Charles E. Leiserson. EvolveGCN: Evolving graph convolutional networks for dynamic graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 5363–5370, 2020.
- John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, 1999.

- Tahereh Pourhabibi, Kok-Leong Ong, Boon Hui Kam, and Yee Ling Boo. Fraud detection: A systematic literature review of graph-based anomaly detection approaches. *Decision Support Systems*, 133:113303, 2020.
- Joaquin Quiñero-Candela, Masashi Sugiyama, Anton Schwaighofer, and Neil D. Lawrence. *Dataset Shift in Machine Learning*. MIT Press, 2009.
- Aravind Sankar, Yanhong Wu, Liang Gou, Wei Zhang, and Hao Yang. DySAT: Deep neural representation learning on dynamic graphs via self-attention networks. In *Proceedings of the 13th International Conference on Web Search and Data Mining*, pages 519–527, 2020.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- Daixin Wang, Jianbin Lin, Peng Cui, Quanhui Jia, Zhen Wang, Yanming Fang, Quan Yu, Jun Zhou, Shuang Yang, and Yuan Qi. Semi-supervised graph attention networks for financial fraud detection. In *IEEE International Conference on Big Data*, pages 1–10, 2019.
- Mark Weber, Jie Chen, Toyotaro Suzumura, Aldo Pareja, Edward Law, Amit Dhurandhar, Ching-Yung Mavromatis, Hana Kanezashi, Dhruv Daws, Siddharth Kumur, and Genta Kola. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. In *KDD Workshop on Anomaly Detection in Finance*, 2019.
- Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021.
- Da Xu, Chuanwei Ruan, Evren Korpeoglu, Sushant Kumar, and Kannan Achan. Inductive representation learning on temporal graphs. In *International Conference on Learning Representations*, 2020.
- Jiaxuan You, Tianyu Du, and Jure Leskovec. ROLAND: Graph learning framework for dynamic graphs. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2358–2366, 2022.
- Bianca Zadrozny and Charles Elkan. Obtaining calibrated probability estimates from decision trees and naive Bayesian classifiers. In *Proceedings of the 18th International Conference on Machine Learning*, pages 609–616, 2001.
- Tianxiang Zhao, Xiang Zhang, and Suhang Wang. GraphSMOTE: Imbalanced node classification on graphs with graph neural networks. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, pages 833–841, 2021.
- Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2847–2856, 2018.